



EFREI Paris

Master 2 Data Engineering & IA

RNCP36739 – Expert en Ingénierie de données

Rapport de validations des blocs de compétences

Auteur : Tremont-Raimi Julien & Vandeplanque Antoine
Promotion : 2024–2026
Encadrant : A compléter
Entreprise : A compléter

18 juin 2026

Table des matières

Remerciements	1
1 Introduction	2
2 Présentation d'entreprise	2
2.1 L'entreprise : Geismar	2
2.2 Le poste : Alternant Data Engineer	2
2.2.1 Contexte	2
2.2.2 Missions	2
2.2.3 Environnement technique	3
3 Synthèse des blocs de compétences RNCP36739	3
4 Bloc 1 – Concevoir et développer une architecture de stockage de données	4
4.1 Compétences visées	4
4.2 Projets / TP associés	4
4.3 Présentation des projets mobilisés	4
4.3.1 Challenge Datalake	4
4.4 Justification des compétences du bloc 1	9
4.4.1 Concevoir et développer une base de données relationnelle	9
4.4.2 Concevoir et développer une base de données non relationnelle	9
4.4.3 Concevoir et construire un datalake	10
4.4.4 Créer une API pour rendre les données accessibles	10
4.5 Bilan du bloc 1	10
5 Bloc 2 – Concevoir, développer et deployer une solution de traitement des données massives	11
5.1 Compétences visées	11
5.2 Projets / TP associées	11
5.3 Présentation des projets mobilisés	11
5.3.1 Projet Spark	11
5.3.2 Projet DevOps : fizzbuzz-Efrei	16
5.4 Analyse attendue	17
5.4.1 Preuves d'acquisition des compétences	17
6 Bloc 3 – Implementer et optimiser des solutions de stockage et de traitement de données sur le cloud	18
6.1 Compétences visées	18
6.2 Projets / TP associées	18
6.3 Contexte et enjeux	18
6.3.1 Positionnement dans l'écosystème AWS	18
6.4 Architecture cloud	19
6.5 Stockage S3 — data lake brutes horodaté	20
6.6 DynamoDB — modèle d'accès et index secondaire global	21
6.6.1 Modèle de données	21
6.6.2 Global Secondary Index — requête O(1) sur le dernier snapshot	22
6.6.3 Ingestion incrémentale depuis S3	22
6.7 Traitement cloud — agrégation analytique	23
6.8 Mise à disposition — Lambda et API Gateway	23
6.8.1 Routage des 13 endpoints serverless	23
6.8.2 Cold start et stratégies de mitigation	24
6.9 Sécurité et gestion des identités	24
6.9.1 Principe du moindre privilège (IAM)	24
6.9.2 Chiffrement des données	24
6.9.3 Automatisation de l'ingestion par EventBridge	25
6.10 Optimisation des performances et des coûts	25
6.10.1 Serverless contre serveur permanent	25

6.10.2	Cycle de vie du stockage S3	25
6.10.3	Supervision CloudWatch	25
6.11	Synthèse — compétences démontrées	26
6.11.1	Preuves d'acquisition des compétences	26
7	Bloc 4 – Implementer des methodes d'intelligence artificielle	27
7.1	Compétences visées	27
7.2	Contexte et enjeux	27
7.3	Architecture et cycle de vie du modèle	27
7.4	Analyse exploratoire et compréhension métier	28
7.4.1	Exploration des données	29
7.5	Préparation des données et pipeline de transformation	30
7.5.1	Variables explicatives et cibles	30
7.5.2	Le pipeline de transformation : ColumnTransformer	30
7.6	Entraînement comparatif et sélection du modèle	32
7.6.1	Architecture du benchmark	32
7.6.2	Modèles candidats	32
7.6.3	Le modèle vainqueur : HistGradientBoostingClassifier	32
7.7	Résultats et évaluation	33
7.7.1	Tâche 1 — Classification binaire <code>failure_within_24h</code>	33
7.7.2	Tâche 2 — Classification multiclasse <code>failure_type</code> (5 classes)	33
7.7.3	Interprétabilité — importance des variables	34
7.7.4	Analyse des erreurs — matrice de confusion	35
7.8	Mise en service et communication des résultats	37
7.8.1	API de prédiction — FastAPI	37
7.8.2	Tableau de bord interactif — Streamlit	37
7.8.3	Qualité logicielle et reproductibilité	38
7.9	Robustesse, limites et axes d'amélioration	39
7.9.1	Validation : split stratifié et perspective k-fold	39
7.9.2	Limites de l'interprétabilité par permutation	39
7.9.3	Axes d'amélioration	39
7.10	Synthèse — compétences démontrées	39
7.10.1	Preuves d'acquisition des compétences	39
8	Bloc 5 – Concevoir une strategie de management et de gouvernance de données pour transformer les données en informations creatrices de valeur	40
8.1	Competences visees	40
8.2	Projets / TP associes	40
8.3	Analyse attendue	40
9	Conclusion	41

Remerciements

Je tiens à exprimer mes remerciements les plus sincères au intervenants et professeurs. La richesse de leurs enseignements ont grandement enrichi mes connaissances et affûté mes compétences ; leur encadrement, leur conseils, prodigués tout au long de cette formation, ont été essentiels de ma progression.

Enfin, je voudrais exprimer ma profonde gratitude envers toutes les personnes de Geismar pour le cadre d'apprentissage qu'elles m'ont offert. Cet environnement a joué un rôle déterminant dans mon développement professionnel en me permettant d'acquérir de nouvelles compétences. Je tiens à souligner combien les opportunités d'expérimentation qui m'ont été offertes ont été d'une valeur inestimable.

À tous, merci pour votre contribution essentielle à la concrétisation de ce projet.

1 Introduction

La donnée s'est imposée comme l'un des actifs stratégiques majeurs des organisations contemporaines. Derrière chaque tableau de bord, chaque modèle prédictif et chaque service numérique se trouve une chaîne d'ingénierie qui transforme des flux bruts, massifs et hétérogènes en information fiable, exploitable et créatrice de valeur. C'est précisément ce métier de **data engineer** que ce rapport entend illustrer : concevoir, construire et industrialiser cette chaîne, de la source jusqu'à la décision.

Ce document constitue le dossier écrit de certification professionnelle **RNCP36739** « Expert en Ingénierie de Données », produit dans le cadre du Mastère Data Engineering & IA d'EFREI Paris (promotion 2024–2026). Il vise à démontrer l'acquisition des cinq blocs de compétences du référentiel [1] à travers des **réalisations concrètes** plutôt que des connaissances théoriques isolées.

La démonstration s'appuie sur cinq projets complémentaires, chacun choisi pour illustrer au mieux un pan du métier : **Jobtech**, plateforme d'agrégation de données d'emploi tech à partir de six sources hétérogènes ; Le **project cloud**, plateforme serverless AWS de mobilité urbaine en temps réel ; Le **project spark**, pipeline Big Data distribué sur données aéronautiques ; **fizzbuzz Efrei**, chaîne d'intégration et de déploiement continu conteneurisée ; et le **project data science project**, système de maintenance prédictive industrielle par apprentissage automatique. Le tableau ci-dessous récapitule leur rattachement aux blocs du référentiel.

Projet	Domaine	Blocs couverts
Jobtech	Agrégation de données d'emploi tech européen	Bloc 1 (principal), Bloc 5
project_cloud	Plateforme temps réel de mobilité (AWS)	Bloc 1 (NoSQL), Bloc 3, Bloc 5
projet_spark	Pipeline Big Data aéronautique US	Bloc 2 (principal)
fizzbuzz-Efrei	Chaîne CI/CD et conteneurisation	Bloc 2 (DevOps)
data_science_project	Maintenance prédictive industrielle (ML)	Bloc 4 (principal)

Chaque bloc suit une structure identique — compétences visées, contexte et enjeux, architecture, réalisations techniques commentées, axes d'amélioration et synthèse reliée au référentiel — de sorte que chaque partie soit autonome et directement évaluable. Le rapport est précédé d'une présentation de l'entreprise d'alternance, puis d'une synthèse de la couverture des cinq blocs de compétences.

2 Présentation d'entreprise

2.1 L'entreprise : Geismar

Fondée en 1924 à Colmar par Lucien Geismar, la société conçoit, fabrique et vend dans le monde entier des outils et des machines destinés à la pose et à l'entretien des voies ferrées et des caténaires.

Le siège social est aujourd'hui situé à la Défense. L'entreprise compte 750 salariés, dont 420 en France, et possède cinq usines : trois en France (Colmar, Marseille et Saint-Didier-de-la-Tour) et deux à l'étranger (Italie et États-Unis).

Geismar est présent commercialement dans de nombreux pays, notamment en France, au Royaume-Uni, aux États-Unis, au Brésil, en Allemagne, en Italie, en Espagne, au Canada et en Afrique du Sud. Le groupe réalise environ 85 % de son chiffre d'affaires à l'étranger.

2.2 Le poste : Alternant Data Engineer

2.2.1 Contexte

Geismar utilise différents systèmes ERP selon les filiales : M3, Business Central, ou encore des bases Access héritées de systèmes plus anciens. Cette diversité crée un besoin de centralisation et d'harmonisation des données à l'échelle du groupe.

2.2.2 Missions

Mon rôle consiste à développer des pipelines de données en Python pour extraire, transformer et charger les données issues des différents ERP. Ces données sont ensuite structurées via une solution de big data selon une architecture Bronze-Silver-Gold, qui permet de passer progressivement des données brutes à des données exploitables pour le reporting.

Je travaille principalement sur les données financières, la gestion des stocks, le suivi des commandes et le reporting opérationnel des différentes filiales. Une partie importante de mon travail consiste également à migrer les données vers Snowflake.

2.2.3 Environnement technique

J'utilise principalement Python et SQL pour le développement. Les données sont stockées sur SQL Server et Snowflake. L'infrastructure cloud repose sur Microsoft Azure, notamment Blob Storage pour le stockage et Azure DevOps pour le versionning et les pipelines. Côté restitution, les données alimentent des rapports Power BI.

3 Synthèse des blocs de compétences RNCP36739

La certification « Expert en Ingénierie de Données » s'articule autour de cinq blocs de compétences fondamentaux. Ils couvrent l'intégralité du cycle de vie de la donnée dans un environnement Big Data : de la conception d'architectures de stockage et de traitement des données massives (Blocs 1 et 2) à leur déploiement optimisé sur le cloud (Bloc 3), en passant par l'application des méthodes d'intelligence artificielle (Bloc 4) et l'élaboration de stratégies de gouvernance des données (Bloc 5).

Bloc	Intitulé	Compétences clés
RNCP36739 Bloc 1	Concevoir et développer une architecture de stockage de données	Bases de données relationnelles et non-relationnelles, construction de Datalake, création d'API d'accès aux données.
RNCP36739 Bloc 2	Concevoir, développer et déployer une solution de traitement des données massives	Architectures distribuées, systèmes de streaming, optimisation des pipelines, automatisation CI/CD (conteneurisation, ordonnancement).
RNCP36739 Bloc 3	Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud	Solutions cloud (stockage, pipelines), sécurisation (chiffrement, IAM, conformité), optimisation des performances et des coûts.
RNCP36739 Bloc 4	Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements	Extraction et préparation des données, tableaux de bord interactifs, modèles prédictifs supervisés, évaluation des performances.
RNCP36739 Bloc 5	Concevoir une stratégie de management et de gouvernance de données	Gouvernance (politiques, rôles), conformité (régulations, vie privée), audit qualité, stratégie de sécurité et d'accès aux données.

4 Bloc 1 – Concevoir et développer une architecture de stockage de données

4.1 Compétences visées

- Concevoir et développer une base de données relationnelle en réponse aux besoins d'un client en vue de la mise à disposition de ses données structurées, en utilisant les technologies et les langages de requêtes adaptés aux développements envisagés.
- Concevoir et développer une base de données non-relationnelle en vue de la mise à disposition de données semi-structurées et non structurées pour un traitement analytique ou d'intelligence artificielle, en utilisant les technologies et les langages de requêtes adaptés.
- Concevoir et construire un datalake en choisissant les architectures, les indicateurs de performance et les solutions de stockage appropriées afin d'intégrer des données provenant de systèmes d'information variés.
- Créer une API en utilisant les technologies permettant de rendre les données accessibles et d'augmenter l'efficacité et la praticité des applications et des services.

4.2 Projets / TP associés

- Challenge Datalake 1 Data Integration
- Project Cloud

4.3 Présentation des projets mobilisés

4.3.1 Challenge Datalake

Pour ce bloc, nous nous appuyons principalement sur le projet *Challenge Datalake* réalisé en équipe. L'objectif était de construire une plateforme capable de produire une vision consolidée du marché technologique européen en croisant plusieurs signaux : popularité des technologies, demande des recruteurs, salaires, expérience attendue et évolution temporelle. Le projet ne consistait donc pas seulement à stocker des données, mais à mettre en place une chaîne complète capable de capter des informations hétérogènes, de les fiabiliser puis de les restituer sous une forme exploitable.

Problématique métier. L'idée directrice était de répondre à des questions simples en apparence, mais complexes d'un point de vue data : quelles technologies montent ou reculent, quelles compétences sont les plus demandées, dans quels pays les opportunités sont les plus nombreuses, et quelles tendances peut-on observer entre la popularité d'une technologie et sa présence dans les offres d'emploi. Pour cela, il fallait rapprocher des sources qui n'avaient ni la même granularité, ni le même format, ni le même rythme de mise à jour.

Sources de données mobilisées. Le projet reposait sur six sources principales, chacune apportant une lecture complémentaire du marché.

- *Google Trends* fournissait des séries temporelles sur l'intérêt de recherche associé à différentes technologies.
- *GitHub Repositories* permettait de mesurer la vitalité de l'écosystème open source à travers les dépôts, les langages, l'activité et la popularité des projets.
- *StackOverflow Developer Survey* apportait des données d'enquête structurées sur les technologies utilisées, les profils développeurs, les salaires et les pratiques déclarées.
- *Adzuna Jobs API* donnait accès à des offres d'emploi avec informations de salaire, de localisation et d'intitulé de poste.
- *EuroTechJobs* complétait la couverture du marché européen via des offres récupérées par scraping.
- *Jobicy* ajoutait une dimension *remote-first*, utile pour capter les opportunités de travail à distance.

Cette diversité était un vrai enjeu technique : certaines sources étaient orientées séries temporelles, d'autres jeux de données tabulaires annuels, d'autres encore des flux d'annonces en JSON ou des pages web à parser. Nous avons donc travaillé à la fois avec des API, des fichiers CSV, des structures semi-structurées et des données issues de scraping.

Architecture générale du projet. Pour absorber cette hétérogénéité, nous avons conçu une architecture de type *Medallion* organisée en trois couches, implémentées dans des dossiers techniques distincts du dépôt : `src/bronze`, `src/silver` et `src/gold`. À cela s'ajoutait `src/api` pour l'exposition des données et `src/utils` pour les composants transverses comme la gestion des connexions SQL, des chemins, des schémas et de Spark. Le dépôt contient également un script principal d'orchestration permettant d'exécuter le pipeline couche par couche, ce qui rendait les traitements plus lisibles et plus faciles à rejouer.

Couche Bronze : ingestion et conservation du brut. La couche *Bronze* avait pour rôle de collecter la donnée source sans la transformer fortement. Chaque source disposait de son propre script d'import, comme `import_github_repos.py`, `import_stackoverflow_survey.py`, `import_eurotechjobs.py` ou `import_jobicity.py`. L'idée était de respecter au maximum la structure d'origine afin de conserver une trace fidèle des données récupérées. Cette étape jouait le rôle de zone d'atterrissage du datalake : elle facilitait le debug, la reprise sur incident et le reprocessing, car nous pouvions repartir de la donnée brute sans relancer systématiquement les collectes externes.

Dans la pratique, la couche Bronze n'était pas un simple dossier de dépôt. Elle portait déjà des choix techniques importants : gestion des appels API, temporisation pour limiter les erreurs de quota, sérialisation des données, horodatage des extractions et sauvegarde dans des chemins standardisés. Sur le flux GitHub par exemple, le script d'ingestion interroge l'API par langage ou par thématique, calcule un premier score d'activité, puis écrit le résultat dans la couche Bronze au format Parquet avec un nom de fichier daté. Cela donne une base historisée, exploitable pour des traitements ultérieurs ou pour comparer plusieurs runs d'ingestion.

```
1 df = pd.DataFrame(repos_data)
2 df['created_at'] = pd.to_datetime(df['created_at'])
3 df['year'] = df['created_at'].dt.year
4 df['month'] = df['created_at'].dt.month
5
6 output_path = get_bronze_path('github_repos')
7 output_path.mkdir(parents=True, exist_ok=True)
8
9 filename = get_timestamped_filename("github_repos")
10 full_path = output_path / filename
11
12 df.to_parquet(full_path, engine='pyarrow', compression='snappy', index=False)
```

Listing 1 – Extrait de la couche Bronze : sauvegarde d'une collecte GitHub dans la zone brute.

Cet extrait est représentatif de la philosophie Bronze : conserver un export brut mais structuré, dans un format de fichier performant et relançable, sans encore introduire toute la logique métier de normalisation.

Couche Silver : normalisation et fiabilisation. La couche *Silver* correspondait au moment où la donnée devenait réellement travaillable. Les scripts de transformation de cette couche, dédiés chacun à une source ou à une famille de données, appliquaient des règles de nettoyage et d'harmonisation. Cela incluait le renommage des colonnes, le typage, la standardisation des pays et des dates, la préparation des champs de salaire, la gestion des valeurs manquantes, la suppression des doublons et la mise en cohérence des champs décrivant les technologies. Cette phase avait une importance forte, car elle conditionnait la possibilité de comparer ensuite des données qui, au départ, n'avaient pas le même vocabulaire ni le même niveau de détail.

La couche Silver jouait aussi un rôle de contrôle qualité. Pour certaines sources, les scripts commençaient par relire l'ensemble des fichiers Parquet de la Bronze, les concaténer, puis appliquer des traitements de déduplication et de normalisation. Sur le flux GitHub, le traitement supprime les doublons sur l'identifiant du dépôt, nettoie les noms, recalcule les métriques d'activité, attribue un niveau d'activité et produit un score de qualité de donnée. Le résultat est ensuite chargé dans MySQL avec une logique d'*upsert*, afin de mettre à jour les enregistrements déjà présents sans recréer inutilement toute la table.

```
1 df = df.drop_duplicates(subset=['id'], keep='last').copy()
2 df.loc[:, 'name_cleaned'] = df['name'].str.strip().str.lower()
3 df.loc[:, 'technology_normalized'] = df['technology'].str.strip().str.lower()
4
5 for field in ['stars_count', 'forks_count', 'watchers_count']:
6     df.loc[:, field] = pd.to_numeric(df[field], errors='coerce').fillna(0)
7
8 df = self._recalculate_metrics(df)
9 df.loc[:, 'processed_at'] = pd.Timestamp.now()
```



```
10 df.loc[:, 'data_quality_score'] = self._calculate_quality_score(df)
```

Listing 2 – Extrait de la couche Silver : nettoyage, recalcul et préparation des données GitHub.

Cette étape est centrale dans un projet de datalake, car c'est elle qui fait passer la donnée d'un état « collecté » à un état « exploitable ». Sans cette standardisation intermédiaire, les jointures logiques entre sources et les comparaisons analytiques resteraient trop fragiles.

Couche Gold : consolidation et vision analytique. La couche *Gold* était portée par un travail d'unification, notamment via le module `table_unification.py`. Son objectif n'était plus seulement de nettoyer les données, mais de produire des tables de consommation et des vues analytiques. C'est à ce niveau que les différentes sources étaient rapprochées autour de dimensions communes comme la technologie, le pays, la période ou le type d'indicateur. Cette couche constituait la base des restitutions finales, car elle proposait une information déjà consolidée, plus simple à interroger pour des analyses transverses ou pour une exposition via API.

Techniquement, cette couche se rapprochait d'un entrepôt de données. Le script de unification créait d'abord des tables de dimensions pour le calendrier, les pays, les technologies, les entreprises et les sources de données, puis des tables de faits dédiées à l'activité technologique et aux offres d'emploi détaillées. Cette organisation en étoile avait un intérêt clair : fournir un modèle plus lisible pour l'analyse croisée, où les sources hétérogènes deviennent des métriques comparables autour de dimensions communes.

```
1 'analysis_tech_activity': """
2     CREATE TABLE IF NOT EXISTS analysis_tech_activity (
3         id_activity BIGINT AUTO_INCREMENT PRIMARY KEY,
4         date_key DATE NOT NULL,
5         id_country INT,
6         id_technology INT,
7         id_source SMALLINT,
8         job_count INT DEFAULT 0,
9         avg_salary_usd DECIMAL(10,2),
10        github_stars INT DEFAULT 0,
11        search_volume INT DEFAULT 0
12    ) ENGINE=InnoDB
13 """
```

Listing 3 – Extrait de la couche Gold : création d'une table de faits orientée analyse.

Ce type de structure montre bien la différence entre Silver et Gold. En Silver, on fiabilise les tables source par source. En Gold, on change de niveau de raisonnement : on construit des objets orientés décision, pensés pour l'analyse et non plus seulement pour le nettoyage ou la conservation.

Choix de stockage. L'architecture s'appuyait sur une stratégie de stockage mixte. Les données collectées et les jeux intermédiaires étaient conservés sous forme de fichiers dans les répertoires du datalake, en particulier dans une logique de stockage brut et de retraitement, avec des formats comme JSON, CSV ou Parquet selon les sources et les traitements. En parallèle, les couches structurées *Silver* et *Gold* étaient chargées dans MySQL, avec des bases dédiées. Ce découplage est important : le stockage fichier servait la traçabilité, l'archivage et la souplesse des pipelines, tandis que MySQL apportait une structure relationnelle fiable pour la consultation, les filtres, les recherches et l'exposition des données. Nous avons donc combiné deux logiques complémentaires plutôt que de choisir un seul mode de stockage.

La structure technique globale du projet peut être résumée par le schéma suivant.

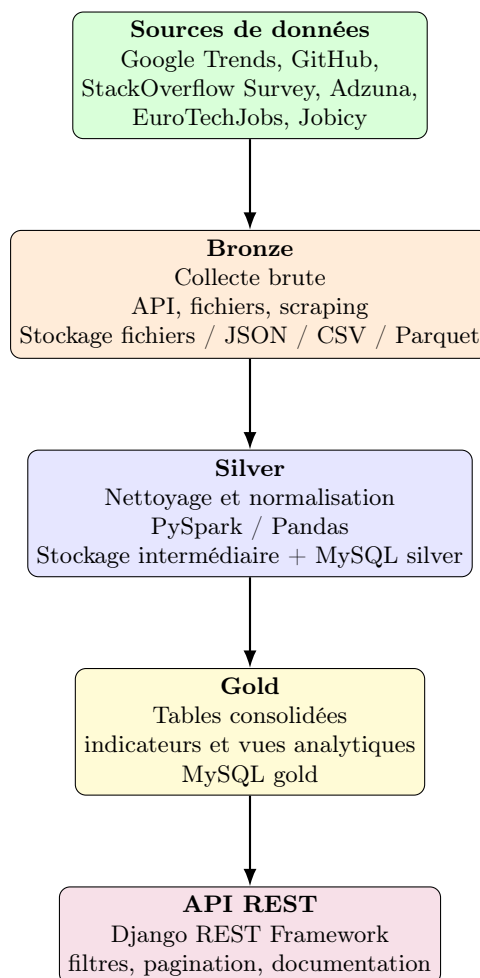


Figure 1 : Synthèse de l'architecture technique du projet Challenge Datalake.

Traitements et outils techniques. Sur la partie traitement, nous avons mobilisé principalement Python, PySpark et Pandas. Python servait de langage commun pour l'ingestion, l'orchestration et les scripts métiers. PySpark intervenait pour traiter des jeux de données plus lourds ou pour structurer les transformations dans une logique proche d'un pipeline big data. Pandas était utilisé sur certains flux lorsque la volumétrie restait raisonnable et qu'un traitement tabulaire classique suffisait. Cette combinaison nous a permis d'adapter l'outil au type de source et au volume traité, tout en gardant une cohérence d'ensemble dans le projet.

Organisation de l'API. Le dernier volet du projet concernait l'exposition de la donnée. Nous avons développé une API REST avec Django et Django REST Framework afin de rendre les résultats accessibles à d'autres applications ou à des utilisateurs métiers. L'API était organisée en modules métiers par source, par exemple `github_jobs`, `stackoverflow_survey`, `eurotechjobs`, `jobicy_jobs`, `trends` et `adzuna_jobs`, ainsi qu'un module `analysis` pour les restitutions transverses. Cette structuration reprenait la logique du pipeline : chaque source avait sa propre porte d'entrée, mais l'API proposait aussi une lecture consolidée du marché.

Fonctionnalités d'exposition. Au-delà de la simple mise à disposition des tables, l'API proposait des fonctions de filtre, de tri, de pagination et de documentation automatique via Swagger / OpenAPI. Le projet comportait également des éléments d'authentification et de gestion des permissions, ce qui montre que l'enjeu n'était pas seulement de sortir des données, mais de construire une couche de service exploitable, documentée et réutilisable. Dans une logique de démonstration, la collection Postman fournie dans le dépôt permettait de tester rapidement les endpoints et de valider les cas d'usage principaux.

Valeur du projet dans le rapport. Ce projet est particulièrement intéressant pour ce rapport, car il constitue un cas concret d'architecture de données de bout en bout. Il relie des problématiques de

collecte, de structuration, de stockage, de transformation et de diffusion dans une même chaîne technique. En d'autres termes, il ne s'agit pas d'un simple exercice d'import de données, mais bien d'une plateforme cohérente où chaque couche répond à un rôle précis, depuis la source brute jusqu'au service exposé aux consommateurs.

Pour la sous-compétence NoSQL, nous mobilisons en complément un autre projet, *project_cloud*, mais uniquement sur son volet NoSQL. Dans ce cadre, nous avons utilisé Amazon DynamoDB pour concevoir une base non relationnelle orientée cas d'usage, destinée à stocker rapidement des données semi-structurées issues de plusieurs flux urbains sans imposer un schéma relationnel rigide.

4.4 Justification des compétences du bloc 1

4.4.1 Concevoir et développer une base de données relationnelle

Le *Challenge Datalake* démontre clairement cette compétence, car la couche *Silver* puis la couche *Gold* reposent sur MySQL pour structurer et exposer les données nettoyées. Dans ce projet, la base relationnelle ne sert pas uniquement de zone de stockage ; elle joue le rôle de *data warehouse* centralisé, destiné à accueillir des données normalisées provenant de plusieurs sources externes.

Le rapport du projet explique que la couche Bronze collecte les données brutes, que la couche Silver les nettoie et les homogénéise, puis que la couche Gold les unifie pour les rendre exploitables par l'API. Cette logique est confirmée dans le dépôt `julientremont/Jobtech`, où la configuration Django et les utilitaires SQL montrent l'usage explicite de MySQL. On y retrouve notamment des schémas SQL dédiés, une gestion de connexions centralisée, ainsi qu'une modélisation orientée analyse, avec des tables relationnelles adaptées aux requêtes métier.

Cette partie répond donc à la compétence attendue, car il ne s'agissait pas seulement d'installer une base SQL, mais bien de concevoir une structure relationnelle cohérente avec le besoin client : centraliser des données sur le marché de l'emploi tech, les rendre interrogeables, les relier entre elles et permettre leur exploitation à travers une application. Le choix de MySQL est également pertinent au regard de la nature des données structurées finales, notamment pour les analyses croisées, les filtres, la pagination et les accès via API.

4.4.2 Concevoir et développer une base de données non relationnelle

Cette sous-compétence est justifiée par un autre projet que nous avons réalisé, *project_cloud*. Dans ce dépôt, la partie qui nous intéresse pour le bloc 1 est la conception de la couche NoSQL avec Amazon DynamoDB. L'objectif n'était pas de reproduire un modèle relationnel, mais de stocker efficacement des données hétérogènes liées à plusieurs domaines métier de la ville de Paris, notamment les parkings, le trafic et les départs de transport.

Le modèle de données adopté est typiquement orienté document. Chaque enregistrement est stocké sous forme d'item JSON dénormalisé, contenant directement les informations utiles au cas d'usage. Par exemple, les données de parkings incluent dans un même item l'identifiant, le nom, l'adresse, les capacités, les tarifs, l'état courant, les coordonnées et l'horodatage d'ingestion ; les données de trafic embarquent les propriétés métiers et les coordonnées du segment ; les données de départs regroupent l'identifiant du trajet, la gare de départ, la destination, le réseau, les horaires et la géolocalisation. Cette organisation est adaptée à DynamoDB, car elle évite les jointures et privilégie un accès direct à des objets déjà prêts à être consommés.

La modélisation repose sur des tables distinctes par grand flux métier, par exemple **parkings**, **traffic** et **departures**. Ce découpage répond à une logique d'usage : chaque table correspond à un type de données interrogé par des traitements et des endpoints spécifiques. Les items partagent néanmoins certaines métadonnées communes, notamment un champ `type`, un champ `ingestion_timestamp` et, selon les cas, des informations de localisation. Ce choix permet d'unifier les traitements techniques tout en conservant la souplesse propre à chaque domaine. Le code montre également l'usage d'écritures en lot avec `batch_writer()`, ce qui est cohérent avec une logique d'ingestion fréquente et de chargement par lots.

Un point important de cette conception est la prise en compte des patrons d'accès. Nous n'avons pas modélisé la base autour de relations théoriques entre entités, mais autour des requêtes réellement attendues. Les scripts et l'API exploitent un index secondaire global de type *GSI* sur les attributs `type` et `ingestion_timestamp` afin de retrouver rapidement le dernier lot ingéré ou les enregistrements correspondant à une fenêtre d'ingestion précise. Cette stratégie est représentative d'une bonne pratique NoSQL : on part des besoins de lecture, puis on construit la structure de stockage et les index en conséquence.

Cette conception répond à la compétence visée, car elle montre que nous avons su adapter le modèle de stockage à des données semi-structurées et à des besoins d'accès rapides, dans une logique orientée exploitation. Le volet NoSQL de *project_cloud* montre concrètement notre capacité à choisir une technologie non relationnelle lorsque la flexibilité du schéma, la dénormalisation, la montée en charge et la rapidité de lecture priment sur les contraintes d'un modèle SQL classique. C'est précisément ce qui était attendu ici : non pas simplement utiliser DynamoDB, mais concevoir une base cohérente avec la nature des données et les usages applicatifs visés.

4.4.3 Concevoir et construire un datalake

Le *Challenge Datalake* est la partie la plus représentative de cette compétence. L'architecture mise en place repose sur une logique *Bronze / Silver / Gold*, explicitement décrite dans le rapport du projet. La couche Bronze sert à collecter les données brutes depuis différentes sources externes et à les stocker localement au format Parquet. La couche Silver applique les traitements de nettoyage, de normalisation et d'enrichissement. Enfin, la couche Gold consolide les données dans un entrepôt exploitable par l'API et par les analyses.

Le choix de cette architecture est pertinent car il permet de séparer clairement les responsabilités : ingestion brute, préparation de la donnée et exposition des données consolidées. Cette organisation facilite la maintenance, la traçabilité et la reprise des traitements. Elle répond aussi à la diversité des systèmes sources : API publiques, fichiers CSV, enquêtes annuelles, scraping web et données issues de plateformes open source.

Le rapport de rendu détaille plusieurs flux : Google Trends avec collecte historique et quotidienne, StackOverflow Survey avec import de fichiers annuels, Adzuna avec appels API mensuels, EuroTech Jobs et Jobicy via scraping, ainsi que GitHub Repositories via API publique. Cette variété de formats et de rythmes d'ingestion illustre bien la capacité à intégrer des données structurées, semi-structurées et non structurées dans une architecture unique. Le stockage en Parquet et le traitement avec PySpark montrent en outre une démarche orientée performance et scalabilité, notamment grâce au partitionnement temporel et à la standardisation progressive des colonnes.

Nous pouvons donc justifier cette compétence en montrant que le projet ne se limite pas à une simple agrégation de fichiers : il s'agit bien d'un datalake conçu pour absorber plusieurs sources, conserver la donnée brute, produire une donnée fiable et la rendre exploitable dans une couche analytique. C'est précisément l'objectif attendu dans ce bloc.

4.4.4 Créer une API pour rendre les données accessibles

La dernière sous-compétence du bloc est également couverte par le *Challenge Datalake* grâce à l'API REST développée avec Django et Django REST Framework. Le rapport du projet consacre une partie complète à cette API et à son rôle de vitrine de données. L'objectif n'était pas seulement d'entreposer les données, mais de les rendre accessibles de manière claire, documentée et sécurisée.

L'API est organisée par source de données, avec des routes dédiées pour Google Trends, StackOverflow Survey, Adzuna, GitHub, EuroTech Jobs et Jobicy, ainsi que des endpoints d'analyse transverses pour la couche Gold. Le dépôt montre également la présence d'une documentation Swagger / OpenAPI, d'une authentification par session et JWT, d'un système de permissions, ainsi que de fonctionnalités de filtrage, tri et pagination. Ces éléments sont importants, car ils traduisent une conception orientée usage et non une simple exposition brute des tables.

Cette API justifie pleinement la compétence attendue : elle permet à des clients ou à d'autres services d'interroger les données selon leurs besoins, de récupérer des résumés, des séries temporelles ou des analyses métier, tout en conservant un cadre technique robuste. Elle améliore donc concrètement l'accessibilité et la praticité des données, ce qui correspond exactement à l'objectif de la sous-partie *API et Web Services*.

4.5 Bilan du bloc 1

Au travers de ces deux projets, nous pouvons justifier l'ensemble des sous-compétences du bloc 1. Le *Challenge Datalake* démontre notre capacité à concevoir une architecture de stockage complète, à centraliser plusieurs flux de données, à structurer une base relationnelle et à exposer les données via une API robuste. Le projet *project_cloud* complète cette démonstration sur la partie NoSQL, avec un cas concret d'usage de DynamoDB pour des données semi-structurées. Dans l'ensemble, ces réalisations montrent notre capacité à choisir les bons modèles de stockage selon les usages, à organiser les flux de données de bout en bout et à mettre les données à disposition de manière exploitable.

5 Bloc 2 – Concevoir, développer et deployer une solution de traitement des données massives

5.1 Compétences visées

- Concevoir, en s'appuyant sur une veille technologique, une architecture distribuée répondant au besoin du client pour traiter les données massives en entreprise en utilisant les technologies de traitement adaptées.
- Implementer un système distribué en utilisant des technologies de streaming identifiées à partir d'une veille pour traiter des données sur une période précise ou en temps quasi réel.
- Transformer les données provenant de différentes sources en prenant en compte leur variété pour faire de l'analytique à l'échelle.
- Optimiser la performance des pipelines en utilisant les techniques d'intégration et de mise en scène adéquates pour le traitement des données massives.
- Automatiser la création, les tests, l'intégration et le déploiement des pipelines de données en utilisant les technologies de conteneurisation et d'ordonnancement pertinentes.

5.2 Projets / TP associés

- Projet Spark (Big Data Frameworks)
- DevOps & MLOps

5.3 Présentation des projets mobilisés

5.3.1 Projet Spark

Pour le bloc 2, nous retenons en priorité le projet *projet_spark*, réalisé dans le cadre du module Big Data Frameworks. Ce projet consistait à mettre en place une pipeline de processing de données avec Apache Spark afin de croiser deux types de données différentes : des données de vols aériens et des données météorologiques. L'objectif général était de produire une base analytique permettant d'étudier l'effet des conditions météorologiques sur les retards de vols, puis de générer des indicateurs directement exploitables.

Contexte et objectif fonctionnel. Le cas d'usage retenu reposait sur un scénario d'analyse des performances du transport aérien. Nous voulions répondre à des questions concrètes : quelles compagnies cumulent les retards les plus importants, dans quelles conditions météo les perturbations sont les plus marquées, et comment relier les événements météo à des vols programmés à une heure et sur une zone donnée. Le projet s'inscrivait donc pleinement dans une logique de traitement de données massives, car il fallait intégrer des volumes importants de données de vols, les synchroniser avec des séries météo horaires et produire des jeux de données prêts pour l'analyse.

Sources de données et variété des flux. Deux grandes familles de sources étaient mobilisées. La première concernait les données de vols, lues depuis des fichiers CSV traités en mode streaming. La seconde concernait les données météo historiques récupérées via l'API Open-Meteo sur plusieurs points géographiques : aéroports majeurs comme Atlanta (ATL), Denver (DEN), Los Angeles (LAX) et Chicago (ORD), mais aussi points intermédiaires sur certaines routes aériennes. Cette conception est intéressante, car elle combine un flux tabulaire volumineux et un flux externe API, avec une granularité horaire et géographique différente. Le projet devait donc gérer à la fois la volumétrie, la variété des schémas et l'alignement temporel.

Architecture distribuée avec Spark. L'ensemble du projet repose sur une architecture autour de Spark, nous avons installé spark en local sur une machine puissante, le point d'entrée le cluster se fait via `spark://localhost:7077`. Les scripts d'ingestion, de streaming, de nettoyage, d'agrégation et de machine learning utilisent tous la même `SparkSession` où nous avons configuré la mémoire et le nombre de cœurs des CPU nécessaires pour le traitement. L'architecture distribuée permet d'exécuter de nombreuses tâches en parallèle. Le dépôt est structuré selon une structure médaillon *Bronze / Silver / Gold*, à laquelle s'ajoutent un entrepôt local pour la couche Bronze, un metastore Hive côté Silver et une sortie MariaDB

pour la partie Gold. Cette organisation matérialise un pipeline distribué complet, où chaque étape a un rôle technique précis.

Le schéma ci-dessous résume l'architecture technique du projet, depuis les deux sources d'entrée jusqu'à l'analyse des données et la prédiction grâce à un modèle de machine learning.

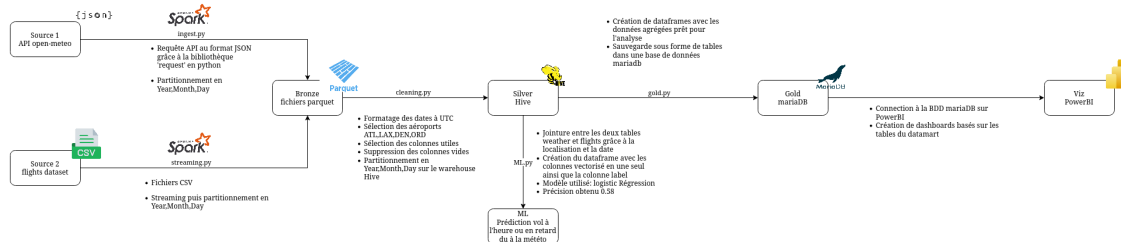


Figure 2 : Vue d'ensemble technique du projet Spark.

Couche Bronze : ingestion batch et streaming. La première étape, la couche *Bronze*, centralise les données brutes et correspond au point d'entrée technique du pipeline distribué. Elle reçoit des flux de natures différentes sans chercher à les harmoniser immédiatement, ce qui permet de conserver une trace fidèle des données réellement collectées. Dans notre projet, cette couche est alimentée par deux mécanismes distincts. Le premier est un traitement batch, porté par `ingest.py`, qui interroge l'API Open-Meteo pour récupérer des historiques météo sur plusieurs localisations puis construit un DataFrame Spark avant de persister les résultats en Parquet. Le second est un traitement streaming, porté par `streaming.py`, qui lit les données de vols depuis des fichiers CSV déposés dans un répertoire source et les intègre progressivement via `readStream`.

Ce choix technique est important, car il montre que la couche Bronze n'est pas seulement un espace de dépôt passif. Elle constitue une zone d'atterrissage capable d'absorber à la fois des extractions historiques et un flux incrémental. Le format Parquet a été retenu pour cette couche brute car il reste compact, colonne par colonne, tout en étant immédiatement exploitable par Spark pour des lectures sélectives. Nous avons également mis en place le partitionnement des vols par `Year`, `Month` et `Day` car il améliore les performances des traitements, puisqu'il permet de limiter les extractions de données à une période donnée au lieu de relire l'ensemble du jeu de données.

Nous avons aussi mis en place des mécanismes de robustesse dans la couche Bronze. Dans le cas du streaming, le *checkpointing* conserve l'état du flux et l'avancement des micro-batches, ce qui permet une reprise plus fiable après interruption sans retraiter arbitrairement tous les fichiers déjà vus. Cette approche est essentielle dans un contexte Big Data, car elle sépare clairement la collecte de la logique d'analyse. En cas d'erreur dans les transformations futures, nous pouvons rejouer les traitements Silver et Gold à partir d'une donnée brute déjà stockée, sans devoir réinterroger l'API météo ni reconstruire manuellement les entrées de vols.

```
1 flights = spark.readStream.format("csv").schema(schema) \
2   .option("header", True).csv("source")
3
4 query = flights_year_month_day.writeStream \
5   .format("parquet") \
6   .partitionBy("Year", "Month", "Day") \
7   .outputMode("append") \
8   .option("path", abspath("bronze/flights")) \
9   .option("checkpointLocation", abspath("bronze/checkpoints/flights")) \
10  .start()
```

Listing 4 – Extrait de la couche Bronze : ingestion streaming des vols vers des fichiers Parquet partitionnés.

Sur la capture d'écran ci-dessous, on voit l'interface Spark Streaming qui permet de suivre l'état des traitements en cours :



Le travail principal de cette couche porte ensuite sur la normalisation temporelle. Les heures de départ et d'arrivée sont d'abord converties en `timestamp`, puis recalculées dans un référentiel commun en UTC à partir des fuseaux horaires des aéroports d'origine et de destination. Cette étape est décisive : sans elle, un vol et une observation météo pourraient être rapprochés alors qu'ils ne correspondent pas au même instant réel. Le projet résout donc une difficulté classique des données massives hétérogènes : rendre comparables des événements issus de systèmes qui n'ont pas la même représentation du temps.

Listing 5 – Extrait de la couche Silver : normalisation des horaires de vol et conversion en UTC.

Le metastore Hive joue ici un rôle important : il conserve les métadonnées des tables, c'est-à-dire leur schéma, leur emplacement physique, leurs partitions et leur nom logique dans le catalogue Spark SQL. Cette couche de métadonnées apporte une différence essentielle avec la Bronze : en Bronze, Spark relit un dossier de fichiers bruts ; en Silver, il peut rouvrir directement des tables structurées et stabilisées, déjà connues du moteur. Ce choix est particulièrement pertinent dans notre projet, car la couche Gold peut s'appuyer sur ces tables intermédiaires pour exécuter ses jointures et ses agrégations sans redéfinir les schémas, sans manipuler explicitement les chemins de fichiers et sans rejouer toute la logique de nettoyage. La persistance Silver constitue donc non seulement un stockage intermédiaire, mais aussi une interface technique propre entre la préparation de la donnée et son exploitation analytique.

Couche Gold : jointures analytiques et restitution. La couche *Gold* s'appuie sur un script Spark SQL dédié. Elle correspond au moment où l'on quitte une logique de préparation technique pour entrer dans une logique de restitution analytique. Les tables Silver de vols et de météo y sont croisées à l'aide de requêtes Spark SQL construites pour rapprocher deux dimensions majeures du projet : la localisation et le temps. Concrètement, la jointure repose sur l'aéroport d'origine du vol et sur une clé horaire dérivée de l'heure de départ, ce qui permet d'associer à chaque vol les conditions météo les plus pertinentes au moment où il est censé décoller.

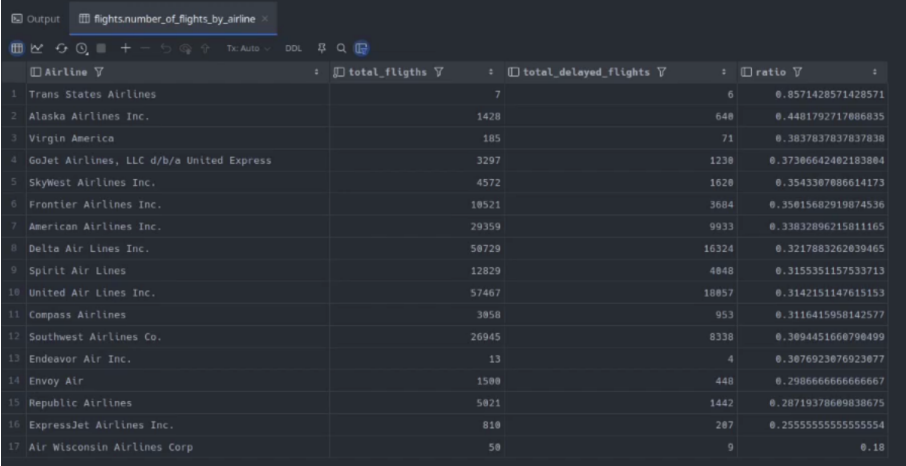
Ce choix de modélisation est central pour la qualité des résultats. Une jointure approximative sur la seule date ou sur une zone géographique trop large aurait produit des indicateurs peu fiables. En imposant une granularité horaire et une correspondance par localisation, le pipeline transforme deux flux indépendants en un jeu d'analyse cohérent. La couche Gold peut alors produire des objets directement consommables : retard moyen par compagnie, impact de la neige ou des précipitations, vols les plus affectés par certaines conditions météo, ou encore ratios de retards par transporteur.

Sur le plan technique, Spark SQL n'apporte pas seulement une écriture plus lisible des requêtes analytiques : il fournit aussi une couche d'exécution unifiée entre le modèle relationnel exprimé par les requêtes et le moteur distribué de Spark. Les jointures entre `flights` et `weather`, ainsi que les agrégations sur les retards, sont formulées sous forme déclarative, puis traduites en plan d'exécution par Spark. Cela permet de bénéficier des optimisations du moteur, notamment sur les projections, les filtres, les phases de jointure et les regroupements, tout en gardant une écriture proche du SQL classique. Dans notre projet, cette approche était particulièrement adaptée, car elle permettait de produire plusieurs tables analytiques spécialisées, comme le retard moyen par compagnie, les vols les plus impactés pendant un épisode neigeux ou les retards maximaux par type de météo, à partir des mêmes tables Silver, sans dupliquer la logique de préparation.

```
1 query = """
2     SELECT f.Airline, f.Origin, f.Dest, f.ArrDelayMinutes,
3           w.snowfall, w.wind_speed_10m, w.precipitation, w.weather_code
4     FROM flights as f
5     LEFT JOIN weather as w
6         ON f.Origin = w.location
7         AND f.CRSDepTime_hourly = w.datetimeUTC
8     WHERE w.weather_code IN (71,73,75)
9     ORDER BY f.ArrDelayMinutes DESC
10 """
11 df = spark.sql(query)
```

Listing 6 – Extrait de la couche Gold : jointure analytique entre vols et conditions météo.

Les résultats finaux ne restent toutefois pas dans Spark. Une fois calculés, ils sont exportés vers MariaDB via le connecteur JDBC, ce qui transforme les sorties Gold en tables relationnelles persistantes, directement interrogeables par des outils externes. Cette étape est importante sur le plan architectural : Spark joue le rôle de moteur de calcul, tandis que MariaDB joue le rôle de service layer. Les consommateurs n'ont donc pas besoin d'accéder au cluster Spark, ni de relancer les scripts de traitement, pour exploiter les indicateurs produits. Dans notre cas, cette base MariaDB était hébergée sur un Raspberry Pi, ce qui permettait d'exposer les tables Gold sur le réseau et de les rendre accessibles en ligne depuis d'autres postes.



Airline	total_flights	total_delayed_flights	ratio
Trans States Airlines	7	6	0.8571428571428571
Alaska Airlines Inc.	1428	648	0.4481792717886835
Virgin America	185	71	0.3837837837837838
GoJet Airlines, LLC d/b/a United Express	3297	1238	0.37386642402183804
SkyWest Airlines Inc.	4572	1628	0.3543387886614173
Frontier Airlines Inc.	19521	3684	0.35015682919874536
American Airlines Inc.	29359	9933	0.33832886215811165
Delta Air Lines Inc.	58729	16324	0.3217883262839465
Spirit Air Lines	12829	4848	0.3155351157533713
United Air Lines Inc.	57467	18857	0.3142151147615153
Compass Airlines	3858	953	0.3116415958142577
Southwest Airlines Co.	26945	8338	0.3094451660798499
Endeavor Air Inc.	13	4	0.3076923876923877
Envoy Air	1588	448	0.2986666666666667
Republic Airlines	5821	1442	0.2871937868938675
ExpressJet Airlines Inc.	810	287	0.25555555555555554
Air Wisconsin Airlines Corp	58	9	0.18

Figure 4 : Exemple de table dans MariaDB

Ce choix a ensuite facilité l'intégration avec Power BI. Plutôt que de connecter l'outil de visualisation directement aux scripts Spark ou aux fichiers intermédiaires, nous avons pu brancher Power BI sur des tables déjà calculées, structurées et stabilisées dans MariaDB. Cette séparation entre calcul distribué et restitution présente plusieurs avantages : elle simplifie la consommation des données, réduit le couplage avec l'environnement Spark, et permet à un outil BI de travailler sur une source relationnelle standard, accessible à distance. Autrement dit, la couche Gold ne produit pas uniquement des résultats analytiques ; elle alimente aussi une interface de diffusion exploitable pour la visualisation, le reporting et la consultation en ligne.

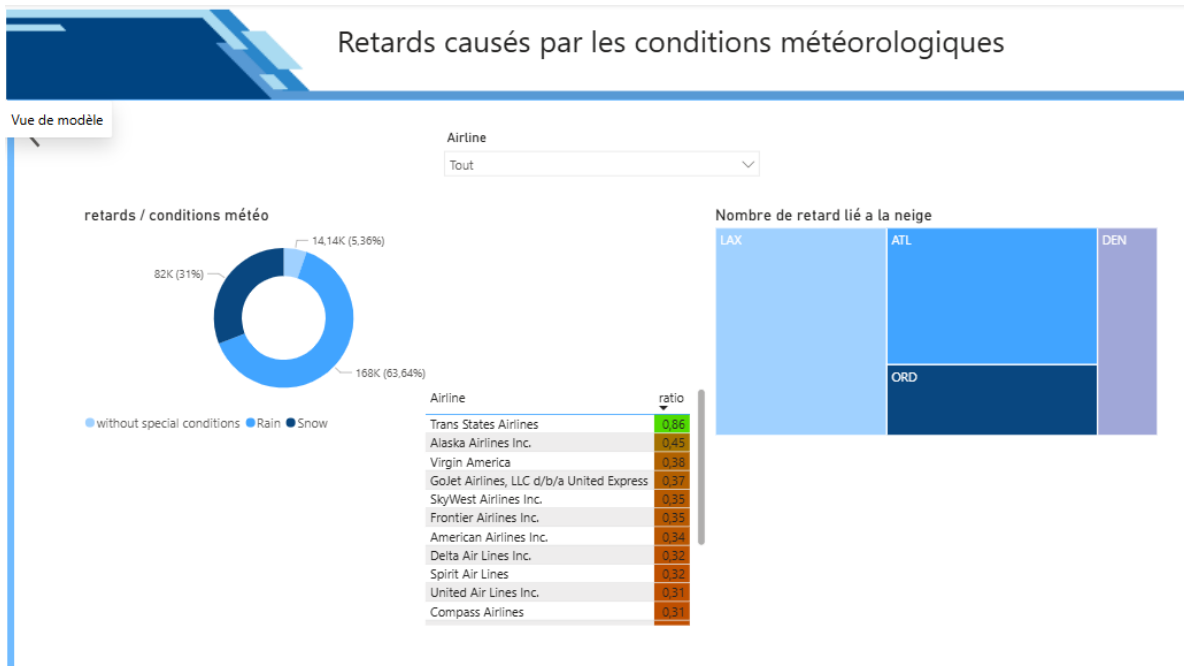


Figure 5 : Dashboard Power BI

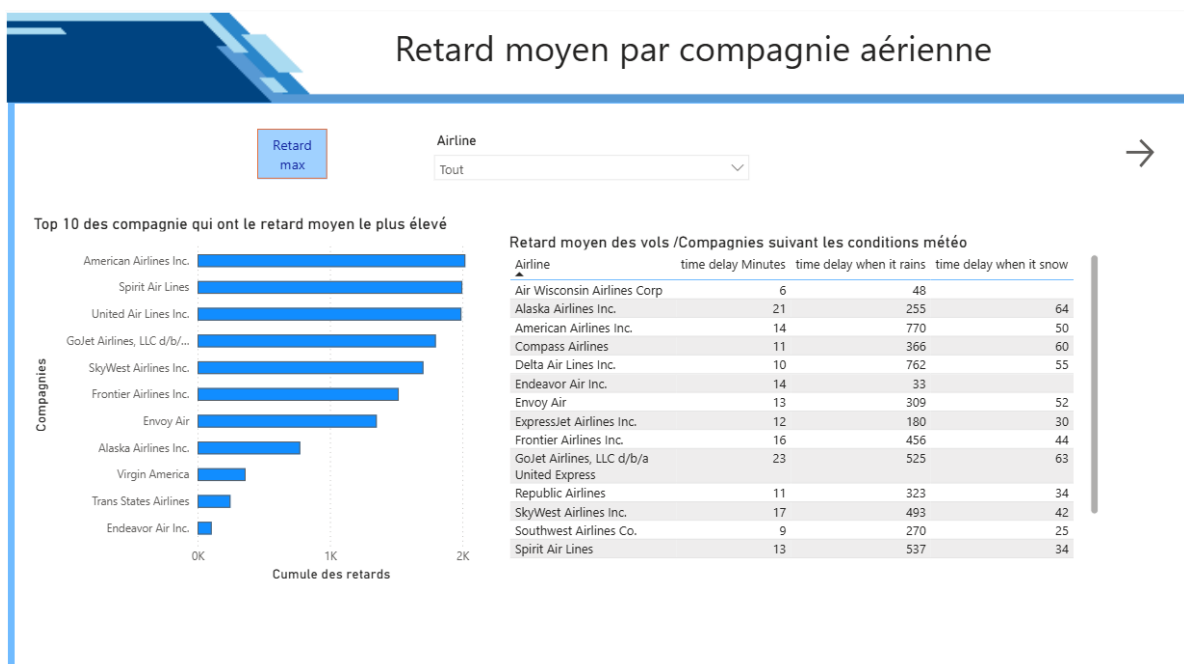


Figure 6 : Dashboard Power BI

Dimension machine learning. Le projet inclut également un script `ML.py`, qui prolonge la chaîne de traitement par une étape de modélisation avec `pyspark.ml`. Les données de vols et de météo y sont fusionnées, transformées en variables numériques, indexées, standardisées puis injectées dans une régression logistique avec validation croisée. Cette partie n'est pas le coeur de la démonstration du bloc 2, mais

elle montre que la chaîne Spark construite est suffisamment robuste pour servir ensuite à des traitements avancés sur données préparées.

5.3.2 Projet DevOps : fizzbuzz-Efrei

En complément du projet Spark, nous mobilisons également le dépôt **fizzbuzz-Efrei** pour justifier la dimension DevOps de ce bloc. Le périmètre fonctionnel de l'application est volontairement simple : il s'agit d'une implémentation Python du problème FizzBuzz accompagnée de tests unitaires. Ce choix est justement intéressant dans le cadre du bloc 2, car il permet de concentrer la démonstration non pas sur la complexité métier, mais sur l'industrialisation du cycle de livraison : contrôle qualité, intégration continue, conteneurisation et préparation au déploiement.

Le premier apport du projet concerne la chaîne d'intégration continue mise en place avec GitHub Actions. Le dépôt contient un workflow `python-package.yml` déclenché sur les branches `dev`, `main` et `docker`. À chaque push, ce pipeline exécute automatiquement la même suite de vérifications sur une matrice de versions Python 3.9, 3.10 et 3.11. Les étapes enchainent l'installation des dépendances, l'analyse statique avec `flake8`, l'exécution des tests unitaires via `python -m unittest test.py`, puis la mesure de couverture avec `coverage run` et `coverage report`. Cette organisation montre que nous avons mis en place une validation systématique du code avant intégration, afin de détecter rapidement les régressions, de garantir la compatibilité sur plusieurs environnements Python et de fiabiliser les livraisons.

Le second apport majeur du projet est la conteneurisation de l'application. Le `Dockerfile` s'appuie sur une image `python:3.12.8-slim`, définit un répertoire de travail dédié, installe les dépendances depuis `requirements.txt` et exécute l'application avec un utilisateur non privilégié. Ce dernier point est important, car il traduit une prise en compte des bonnes pratiques de sécurité dès la phase de packaging. Le dépôt contient également un fichier `compose.yaml` qui permet de reconstruire puis de lancer localement le service avec `docker compose up -build`. Nous obtenons ainsi un environnement d'exécution reproductible, indépendant de la machine du développeur, ce qui réduit fortement les écarts entre développement, validation et livraison.

Le projet ne montre pas un déploiement de production complet avec orchestrateur ou infrastructure cloud, et il ne faut pas le présenter comme tel. En revanche, il montre très clairement la mise en place des briques attendues dans une démarche CI/CD : une application validée automatiquement, empaquetée sous forme d'image Docker et prête à être publiée dans un registre puis déployée dans un environnement cible. Le `README.Docker.md` documente d'ailleurs explicitement les commandes de build et de push d'image. Dans le cadre du bloc 2, cette réalisation est pertinente car elle démontre notre capacité à automatiser la vérification du code, à standardiser l'exécution de l'application et à préparer une chaîne de déploiement outillée, complémentaire des enjeux de traitement massif couverts par Spark.

5.4 Analyse attendue

Mettez en avant la conception des pipelines, le traitement distribué, les choix de performance, l'automatisation et la manière dont vous avez déployé ou industrialisé les traitements.

5.4.1 Preuves d'acquisition des compétences

Le tableau suivant récapitule, pour chacune des cinq sous-compétences du Bloc 2, la réalisation concrète qui la démontre dans le projet.

Compétence visée	Réalisation et preuve dans le projet
Architecture distribuée pour le traitement de données massives	Architecture Spark distribuée structurée en couches Bronze / Silver / Gold, pilotée par une <code>SparkSession</code> commune et un point d'entrée cluster <code>spark://localhost:7077</code> .
Système distribué de streaming	Ingestion incrémentale des vols via <code>readStream</code> , écriture en mode <code>append</code> , partitionnement des sorties et <i>checkpointing</i> pour la reprise fiable des micro-batches.
Transformation de données variées pour l'analytique à l'échelle	Croisement de deux sources hétérogènes (vols CSV et météo API), normalisation temporelle en UTC en Silver, puis jointures analytiques en Gold avec Spark SQL.
Optimisation des performances des pipelines	Utilisation du format Parquet, partitionnement par <code>Year/Month/Day</code> , séparation Bronze / Silver / Gold et exécution distribuée Spark SQL pour limiter les lectures inutiles et accélérer les agrégations.
Automatisation, tests, intégration et déploiement des pipelines	Workflow GitHub Actions exécuté sur plusieurs branches et versions Python, combinant linting <code>flake8</code> , tests <code>unittest</code> , couverture <code>coverage</code> , puis conteneurisation avec Docker et exécution reproductible via <code>docker compose</code> pour préparer la livraison de l'application.

6 Bloc 3 – Implementer et optimiser des solutions de stockage et de traitement de données sur le cloud

Cette partie démontre l'acquisition des quatre compétences du Bloc 3 du référentiel RNCP36739 [1] à travers une plateforme **cloud serverless AWS**. Chaque étape couvre une ou plusieurs sous-compétences du référentiel, synthétisées dans le tableau de validation à la fin du bloc.

6.1 Compétences visées

Le référentiel RNCP36739 définit quatre sous-compétences pour le Bloc 3, que ce dossier couvre intégralement à travers le **projet cloud**. Ce bloc porte sur la mise en œuvre de solutions cloud pour le stockage, le traitement et la mise à disposition des données : choix des services managés, conception des flux, sécurisation des accès, optimisation de la performance et de la disponibilité

- Mettre en œuvre des solutions de stockage de données dans le cloud pour permettre aux entreprises d'explorer leurs données en implémentant des techniques et stratégies adaptées à leur utilisation.
- Concevoir et développer des pipelines et des solutions de traitement de données dans le cloud en chargeant les données, en les transformant et en les mettant à disposition des utilisateurs.
- Mettre en place une politique de sécurité des données dans le cloud en développant une stratégie de chiffrement et de gestion des identités et des accès, dans le respect des règlements en vigueur.
- Optimiser les solutions de stockage et de traitement des données dans le cloud en définissant des indicateurs de performance pour assurer la disponibilité des services et optimiser les coûts.

6.2 Projets / TP associés

- Projet Cloud
- Sécurité des données

6.3 Contexte et enjeux

Le **projet cloud** est une plateforme de données temps réel lyonnaise. Elle agrège en continu trois flux de données ouvertes : les départs ferroviaires et de métro en gare de Lyon (SNCF/RATP), l'état des parkings parisiens (disponibilité, places, tarifs) et l'état du trafic routier (indice de congestion, segments critiques, fluidité par zone). L'objectif est de permettre un suivi opérationnel de la mobilité lyonnaise via un tableau de bord Streamlit alimenté par une API serverless.

L'enjeu architectural central est double : garantir la latence sur des données en temps réel et maîtriser les coûts. Un scan DynamoDB complet pour chaque requête API serait fonctionnellement correct, mais son coût de lecture croîtrait linéairement avec le volume. La mise en place d'un Global Secondary Index (GSI) ramène le coût de chaque requête à $O(1)$, quelle que soit la taille de la table.

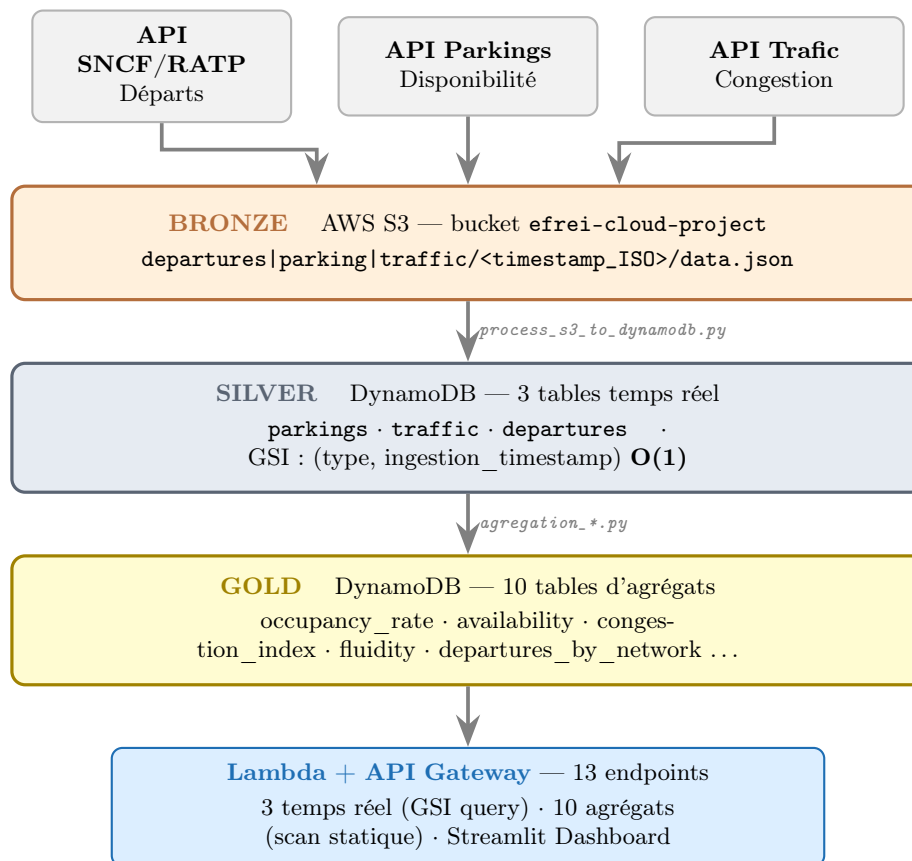
6.3.1 Positionnement dans l'écosystème AWS

AWS propose plusieurs alternatives pour chaque composant de l'architecture. Les choix retenus sont justifiés par rapport à ces alternatives.

Besoin	Choix retenu	Alternative	Justification
Lac de données	S3	EFS, EBS	Coût minimal, accès objet universel
Base NoSQL	DynamoDB	MongoDB Atlas, ElasticSearch	Managé, serverless, GSI natif
Compute API	Lambda	EC2, ECS/Fargate	Pas d'infra à gérer, facturation à l'usage
Routage API	API Gateway	ALB, CloudFront	Intégration Lambda native, gestion CORS
Dashboard	Streamlit (EC2)	QuickSight, PowerBI	Open source, Python natif
Scheduling	EventBridge	Step Functions, MWAA	Cron simple, coût négligeable

6.4 Architecture cloud

L'architecture suit le motif Medallion (Bronze/Silver/Gold) transposé sur des services managés AWS : S3 comme data lake brutes, DynamoDB comme couche nettoyée puis agrégée, et une exposition serverless via Lambda et API Gateway.



Le schéma technique du projet ci-dessous détaille le flux complet, du calcul ordonnancé sur EC2 jusqu'à la visualisation Streamlit.

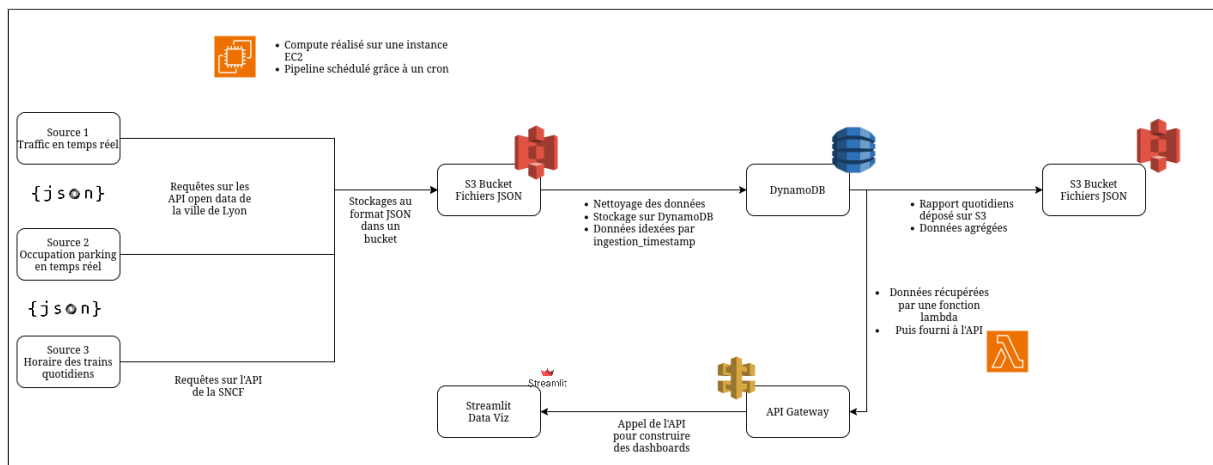


FIGURE 1 – Architecture technique du **projet cloud**. Les trois sources open data sont collectées par un pipeline ordonnancé sur EC2, stockées en JSON dans S3, nettoyées et indexées dans DynamoDB, agrégées dans un rapport quotidien, puis exposées via une fonction Lambda et une API Gateway consommée par le dashboard Streamlit.

6.5 Stockage S3 — data lake brutes horodaté

AWS S3 [?] constitue la couche Bronze de l'architecture. Chaque ingestion produit un objet JSON horodaté dans le préfixe correspondant à la source (`departures/<timestamp_ISO>/data.json`). Cette organisation préserve l'historique complet des snapshots et permet de rejouer n'importe quelle fenêtre temporelle sans perte.

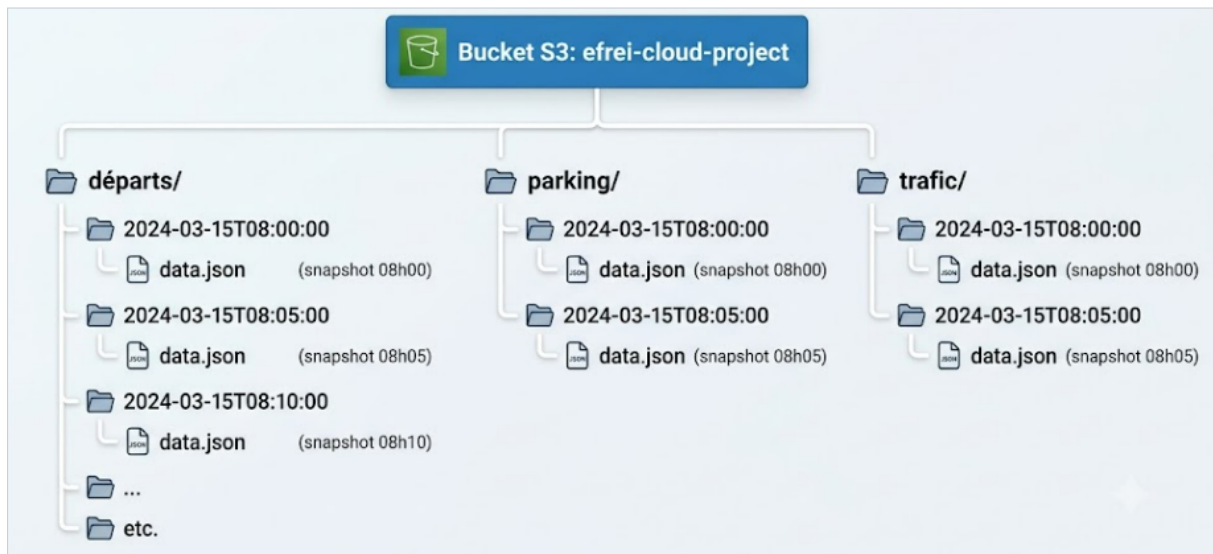


FIGURE 2 – Organisation du bucket S3 *efrei-cloud-project* : un préfixe par source (*départs*, *parking*, *trafic*) et, sous chacun, un dossier horodaté par snapshot contenant le fichier *data.json*.

```

1 def upload_to_s3(s3, json_data, bucket_name, key):
2     s3.put_object(
3         Bucket=bucket_name,
4         Key=key,
5         Body=json.dumps(json_data, indent=4, cls=DecimalEncoder)
6     )
7
8 def get_json_from_s3(s3, bucket, key):
9     response = s3.get_object(Bucket=bucket, Key=key)
10    content = response["Body"].read().decode("utf-8")
11    return json.loads(content)
12
13 def get_s3_object_keys(s3, bucket_name, prefix):
14    response = s3.list_objects_v2(Bucket=bucket_name, Prefix=prefix)
15    return [obj['Key'] for obj in response.get('Contents', [])]

```

Listing 7 – Écriture et lecture S3 (*utils_s3.py*)

Le `DecimalEncoder` sérialise les `Decimal` Python en `int` ou `float` selon la présence d’une partie décimale — une nécessité, car `DynamoDB` stocke tous les nombres au format `Decimal`.

*[Capture] Console AWS S3 – bucket **efrei-cloud-project**, dossier **departures** : liste des objets JSON horodatés.*

6.6 DynamoDB — modèle d’accès et index secondaire global

6.6.1 Modèle de données

Chaque item `DynamoDB` [?] dans les trois tables temps réel porte un attribut `type` (valeur constante par table : `"parkings"`, `"traffic"`, `"departures"`) et un `ingestion_timestamp` en microsecondes Unix. Ce couple constitue la clé du GSI. L’horodatage est exprimé en microsecondes (`timestamp() * 1_000_000`) pour garantir l’unicité sur des ingestions rapprochées : un horodatage en secondes provoquerait des collisions si deux snapshots arrivaient dans la même seconde.

```

1 def create_stations_departures_dict(json_data, ingestion_timestamp_str):
2     results = []
3     for index, departure in enumerate(json_data["departures"]):
4         result = {
5             "gid": index,
6             "trip_id": departure["display_informations"]["trip_short_name"],
7             "ingestion_timestamp": int(
8                 datetime.fromisoformat(ingestion_timestamp_str).timestamp()
9                 * 1_000_000), # microsecondes : unicite garantie
10            "departure_datetime": departure["stop_date_time"]["base_departure_date_time"]
11        ],

```



```

11     "departure_station": departure["stop_point"]["name"],
12     "network":          departure["display_informations"]["network"],
13     "type": "departures",      # PK du GSI : valeur constante par table
14     "long": departure["stop_point"]["coord"]["lon"],
15     "lat":  departure["stop_point"]["coord"]["lat"],
16 }
17 results.append(result)
18 return results

```

Listing 8 – Création d'un item avec horodatage microseconde (process_s3_to_dynamodb.py)

6.6.2 Global Secondary Index — requête O(1) sur le dernier snapshot

L'enjeu de performance est d'accéder au dernier snapshot *sans* scanner toute la table. Sans index, chaque requête « dernier état des parkings » serait un *scan* en O(n) sur l'ensemble de l'historique. Avec le GSI `gsi-type-ingestion_timestamp` (PK : type, SK : ingestion_timestamp), une *query* avec `ScanIndexForward=False`, `Limit=1` retourne l'enregistrement le plus récent en O(1).

Approche	Sans GSI (scan)	Avec GSI (query)
Items lus	Toute la table (N)	Exactement 1
Unités de lecture (RCU)	N	1
Complexité	O(n)	O(1)
Tri	Côté application	Natif (sort key)

```

1 def get_last_processed_timestamp(dynamodb, table_name, type_table):
2     table = dynamodb.Table(table_name)
3     try:
4         response = table.query(
5             IndexName='gsi-type-ingestion_timestamp',
6             KeyConditionExpression=Key('type').eq(type_table),
7             ScanIndexForward=False,  # tri SK descendant : le plus grand en tete
8             Limit=1,
9             ProjectionExpression="ingestion_timestamp"
10        )
11        items = response.get('Items', [])
12        return items[0]['ingestion_timestamp'] if items else None
13    except Exception:
14        # Fallback : scan si GSI non disponible (degraded mode)
15        response = table.scan(
16            ProjectionExpression="ingestion_timestamp", Limit=1000)
17        timestamps = [item['ingestion_timestamp'] for item in response['Items']]
18        return max(timestamps) if timestamps else None

```

Listing 9 – Requête GSI – dernier timestamp O(1) avec fallback scan (utils_dynamodb.py)

Le fallback sur *scan* garantit la disponibilité du service même si le GSI est en cours de construction ou temporairement indisponible (mode dégradé). Ce même pattern — *query* GSI en priorité, *scan* en secours — est réutilisé dans le Lambda de l'API.

*[Capture] Console AWS DynamoDB – table parkings, onglet Indexes : configuration du GSI
gsi-type-ingestion_timestamp.*

6.6.3 Ingestion incrémentale depuis S3

Le pipeline S3 → DynamoDB ne retraits que les nouveaux objets S3. La borne de reprise est le dernier *ingestion_timestamp* connu via le GSI; les clés S3 sont filtrées par comparaison ISO de leur horodatage.

```

1 last_processed_timestamp_str = get_last_processed_timestamp(
2     dynamodb, DYNAMODB_TABLE_NAME, "departures")
3
4 keys = get_s3_object_keys(s3, BUCKET_NAME, s3_folder)
5 new_keys = []
6
7 if last_processed_timestamp_str:
8     last_dt = datetime.fromtimestamp(
9         float(last_processed_timestamp_str) / 1_000_000)
10    for key in keys:

```

```

11     key_dt = datetime.fromisoformat(key.split('/')[1])
12     if key_dt > last_dt:
13         new_keys.append(key)
14 else:
15     new_keys = keys    # premiere ingestion : tout traiter
16
17 for key in new_keys:
18     response = get_json_from_s3(s3, BUCKET_NAME, key)
19     items     = create_stations_departures_dict(response, key.split('/')[1])
20     batch_put_items_to_dynamodb(dynamodb, DYNAMODB_TABLE_NAME, items)

```

Listing 10 – Ingestion incrémentale S3 vers DynamoDB (process_s3_to_dynamodb.py)

L'idempotence est garantie : à chaque relance, seuls les objets S3 postérieurs au dernier horodatage connu sont réingérés, sans duplication des données déjà présentes.

6.7 Traitement cloud — agrégation analytique

Un pipeline d'agrégation lit les tables temps réel (Silver) et produit dix tables analytiques pré-agrégées dans DynamoDB (Gold). Pour les parkings, l'agrégation se fait par tranche horaire (groupement sur `ingestion_timestamp // 3_600_000_000`), puis calcul des indicateurs métier.

Table DynamoDB (Gold)	Indicateur
<code>aggregation_overall_occupancy_rate</code>	Taux d'occupation global (%)
<code>aggregation_average_availability_parking</code>	Disponibilité moyenne par parking
<code>aggregation_reference_pricing</code>	Tarification moyenne (1h/2h/4h/24h)
<code>aggregation_number_of_parkings_in_operation</code>	Nombre de parkings ouverts
<code>aggregation_traffic_congestion_index</code>	Indice de congestion du trafic
<code>aggregation_traffic_critical_segments</code>	Segments routiers critiques
<code>aggregation_traffic_fluidity_by_zone</code>	Fluidité par zone géographique
<code>aggregation_departures_by_network</code>	Départs par réseau (SNCF/RATP)
<code>aggregation_departures_top_destinations</code>	Top destinations
<code>aggregation_departures_total</code>	Total des départs sur la période

Ce pré-calcul transfère la charge CPU du moment de la requête vers le moment de l'ingestion : l'API lit ensuite ces tables d'agrégats (petites et statiques) sans recalculer quoi que ce soit à chaque appel.

6.8 Mise à disposition — Lambda et API Gateway

6.8.1 Routage des 13 endpoints serverless

L'API expose treize endpoints via un Lambda [?] unique routé par chemin : trois endpoints temps réel (requête GSI vers le dernier snapshot) et dix endpoints d'agrégats (scan de tables statiques).

```

1 def lambda_handler(event, context):
2     dynamodb = boto3.resource('dynamodb')
3     path      = event.get('path')
4
5     # Endpoints agregats : scan complet (tables fixes, volume faible)
6     aggregation_tables = [
7         "aggregation_overall_occupancy_rate",
8         "aggregation_average_availability_parking",
9         "aggregation_reference_pricing",
10        "aggregation_traffic_congestion_index",
11        "aggregation_departures_by_network",
12        # ... 10 tables au total
13    ]
14    if path.lstrip('/') in aggregation_tables:
15        items = get_all_items(dynamodb, path.lstrip('/'))
16        return {'statusCode': 200,
17                'body': json.dumps(items, default=decimal_serializer)}
18
19    # Endpoints temps reel : requete GSI -> dernier snapshot
20    if path == '/parkings':    table_name, type_value = "parkings",    "parkings"
21    elif path == '/traffic':   table_name, type_value = "traffic",     "traffic"
22    elif path == '/departures': table_name, type_value = "departures", "departures"
23    else:
24        return {'statusCode': 404,

```

```

25         'body': json.dumps({'message': f'Not Found {path}'})
26
27     last_ts = get_last_ingestion_time(dynamodb, table_name, type_value)
28     items = get_items_by_ingestion_time(dynamodb, table_name, last_ts, type_value)
29     return {'statusCode': 200,
30           'body': json.dumps(items, default=decimal_serializer)}

```

Listing 11 – Lambda handler – routage des 13 endpoints (api/main.py)

L'architecture serverless (Lambda + API Gateway [?]) supprime toute gestion d'infrastructure : pas de serveur à provisionner, pas de scaling manuel, et une facturation à la requête — optimale pour un flux temps réel à appels intermittents.

6.8.2 Cold start et stratégies de mitigation

Une invocation Lambda comprend deux phases : l'initialisation (*cold start*) et l'exécution. Le cold start n'a lieu qu'à la première invocation après une période d'inactivité ; pour un Lambda Python avec boto3, il est typiquement de 200 à 500 ms.

Phase	Durée typique	Fréquence
Cold start (init)	200–500 ms	Première invocation, après inactivité
Warm execution	10–100 ms	Invocations suivantes (même conteneur)

Pour un tableau de bord rafraîchi toutes les 5 minutes, le cold start est acceptable. Pour une API à très faible latence, trois leviers existent : la **Provisioned Concurrency** (instances maintenues chaudes en permanence), un **warm-up ping** EventBridge, ou un package minimal (boto3 étant fourni par le runtime AWS, le ZIP ne contient que le code applicatif ce qui réduit le temps de chargement).

[Capture] Console AWS API Gateway – liste des 13 routes configurées (méthode GET + intégration Lambda).

6.9 Sécurité et gestion des identités

6.9.1 Principe du moindre privilège (IAM)

Chaque composant s'exécute sous un rôle IAM dédié n'accordant que les permissions strictement nécessaires à sa fonction.

Rôle IAM	Permissions	Justification
lambda-api-role	dynamodb:GetItem dynamodb:Query dynamodb:Scan	L'API ne fait que lire — aucune permission d'écriture accordée.
ingestion-role	s3:PutObject dynamodb:PutItem dynamodb:BatchWriteItem	L'ingestion écrit S3 et DynamoDB, sans relecture.
aggregation-role	dynamodb:Query (Silver) dynamodb:PutItem (Gold)	Isolation stricte : lecture Silver, écriture Gold.

Cette séparation garantit qu'une compromission d'un Lambda d'agrégation ne peut ni écrire dans les tables Silver ni lire d'autres tables Gold que les siennes. En production, les rôles seraient affinés par préfixe de table via des conditions IAM **StringLike** sur l'ARN.

6.9.2 Chiffrement des données

Le chiffrement en transit est assuré par défaut en HTTPS (TLS 1.2+) sur tous les appels boto3. Le chiffrement au repos est natif sur chaque service managé.

Service	Mode de chiffrement	Implémentation projet
AWS S3	SSE-S3 (clé gérée AWS)	Chiffrement côté serveur automatique
AWS DynamoDB	AES-256 (clé AWS)	Activé par défaut sur toutes les tables
API Gateway	TLS 1.2+ obligatoire	Aucun trafic HTTP non chiffré

Pour des données sensibles (positions GPS, données personnelles de transport), le SSE-KMS avec rotation annuelle de clé serait retenu : il permet de révoquer l'accès en révoquant la clé, sans supprimer les objets.

6.9.3 Automatisation de l'ingestion par EventBridge

L'ingestion manuelle actuelle serait industrialisée par AWS EventBridge Scheduler : un schedule cron déclenche chaque Lambda d'ingestion toutes les 5 minutes.

```
1 aws scheduler create-schedule \
2   --name "ingest-parkings-every-5min" \
3   --schedule-expression "rate(5 minutes)" \
4   --target '{
5     "Arn": "arn:aws:lambda:eu-west-1:ACCOUNT:function:ingest-parkings",
6     "RoleArn": "arn:aws:iam::ACCOUNT:role/scheduler-invoke-role"
7   }' \
8   --flexible-time-window '{"Mode": "OFF"}'
```

Listing 12 – EventBridge Scheduler – déclenchement périodique (AWS CLI)

Le rôle `scheduler-invoke-role` ne dispose que de la permission `lambda:InvokeFunction` sur la fonction ciblée — nouvelle application du moindre privilège.

6.10 Optimisation des performances et des coûts

6.10.1 Serverless contre serveur permanent

L'architecture Lambda + API Gateway est coût-efficace pour un flux intermittent. La comparaison avec une instance EC2 permanente illustre le gain.

Critère	Lambda + API Gateway	EC2 t3.small
Facturation	À la requête (0,20 \$/M appels)	À l'heure (≈ 18 \$/mois)
Scaling	Automatique, instantané	Manuel ou auto-scaling (délai)
Coût en veille	0 \$	18 \$/mois (actif 24/7)
Maintenance OS	Aucune (managé)	Patches, mises à jour, supervision

Pour le profil du projet (appels toutes les 5 minutes), la facturation Lambda est négligeable : $\approx 8\,640$ invocations/mois, très en dessous du palier gratuit AWS (1 million d'invocations/mois).

6.10.2 Cycle de vie du stockage S3

Le bucket accumule des snapshots toutes les 5 minutes ; sans politique de rétention, le coût de stockage croît indéfiniment. Une politique de cycle de vie S3 automatise l'archivage puis la suppression.

Règle	Délai	Effet
Transition vers S3 Infrequent Access	30 jours	−40 % de coût
Transition vers S3 Glacier	90 jours	−70 % (récupération lente)
Suppression automatique	365 jours	Coût nul

6.10.3 Supervision CloudWatch

CloudWatch collecte automatiquement les métriques de chaque service. Une alarme sur `ThrottledRequests` > 0 déclenche une notification SNS, permettant de réagir avant que le throttling n'affecte les utilisateurs.

Service	Métrique	Seuil d'alerte
Lambda ingestion	Errors	> 0 erreur en 5 min
Lambda API	Duration	> 3 secondes
DynamoDB	ThrottledRequests	> 0 (critique)
API Gateway	5XXError	> 1 % du trafic
S3	NumberOfObjects	Croissance anormale

[Capture] Tableau de bord Streamlit – page Parkings : taux d’occupation global en temps réel et carte des parkings disponibles.

6.11 Synthèse — compétences démontrées

Le projet cloud couvre l’intégralité du périmètre du Bloc 3 à travers une architecture serverless AWS complète, du data lake brutes à l’API de consommation.

6.11.1 Preuves d’acquisition des compétences

Le tableau suivant récapitule, pour chacune des quatre sous-compétences du Bloc 3, la réalisation concrète qui la démontre dans le projet.

Compétence visée	Réalisation et preuve dans le projet
Stockage de données dans le cloud	Lac de données S3 horodaté (Bronze) et base DynamoDB managée (Silver/Gold), avec choix de services justifié face aux alternatives AWS.
Pipelines de traitement cloud	Ingestion incrémentale idempotente S3 → DynamoDB et pipeline d’agrégation produisant 10 tables analytiques pré-calculées.
Sécurité (chiffrement & IAM)	Rôles IAM au moindre privilège, chiffrement au repos (SSE-S3, AES-256) et en transit (TLS 1.2+), automatisation sécurisée via EventBridge.
Optimisation (performance, disponibilité, coûts)	GSI pour un accès O(1), compute serverless facturé à l’usage, cycle de vie S3 et supervision CloudWatch (métriques + alarmes SNS).

7 Bloc 4 – Implementer des methodes d’intelligence artificielle pour modeliser et predire de nouveaux comportements et usages

Cette partie démontre l’acquisition des cinq compétences du Bloc 4 du référentiel RNCP36739 [1] à travers un projet complet de **maintenance prédictive industrielle**. Chaque étape couvre une ou plusieurs sous-compétences du référentiel, synthétisées dans le tableau de validation ci-dessous.

7.1 Compétences visées

Le référentiel RNCP36739 définit cinq sous-compétences pour le Bloc 4, que ce dossier couvre intégralement à travers le **data science project**. Ce bloc porte sur l’exploitation des données pour produire des modèles prédictifs à valeur métier, du reporting analytique et de la visualisation. Il couvre le cycle complet du Machine Learning : préparation des données, construction de pipelines de transformation, entraînement comparatif de modèles, évaluation rigoureuse et mise en service. Le tableau de preuves d’acquisition présenté en fin de dossier (section 7.10.1) récapitule, pour chacune, la réalisation concrète qui la démontre.

- Extraire des données en provenance de systèmes d’information variés pour les exploiter et les analyser en utilisant les outils professionnels courants.
- Préparer les données en les transformant et en les nettoyant pour faire l’analyse et le reporting selon les besoins des différents métiers.
- Élaborer une communication infographique visuelle en construisant des tableaux de bord interactifs afin de communiquer les résultats d’analyse et d’assurer l’extraction de connaissances en temps réel.
- Développer un modèle prédictif pour identifier de nouveaux comportements et usages en implémentant des algorithmes d’apprentissage automatique supervisés.
- Évaluer la performance d’un modèle de machine learning en analysant ses résultats et en les comparant avec d’autres modèles afin d’implémenter la solution la plus convenable à un cas d’usage.

7.2 Contexte et enjeux

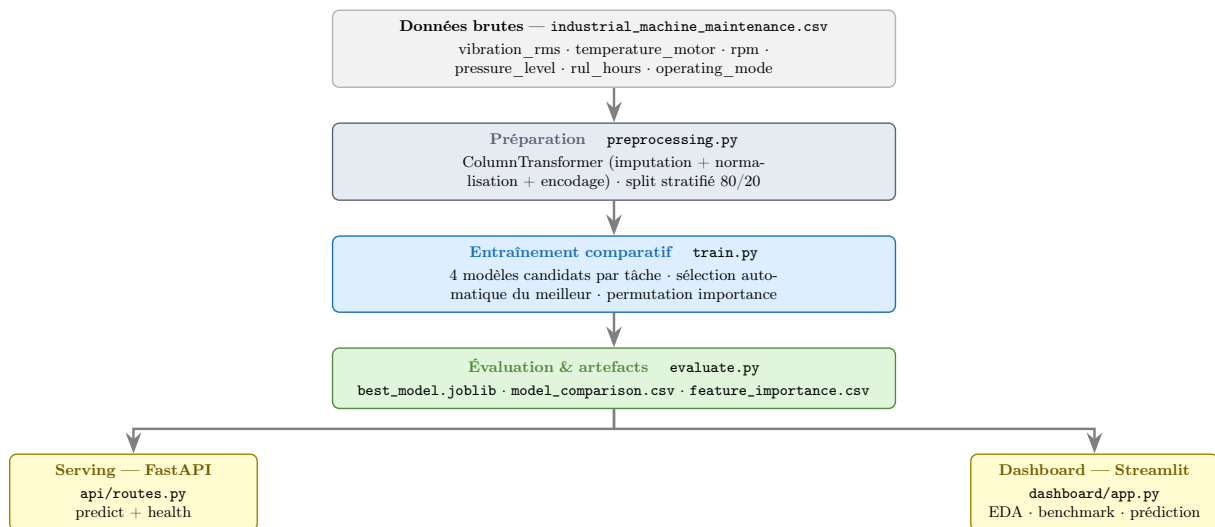
Le projet **data science** traite de la **maintenance prédictive industrielle**. Les machines industrielles sont équipées de capteurs (vibrations, température moteur, vitesse de rotation, pression, heures de vie résiduelle) qui produisent un flux continu de mesures. À partir de ces signaux, nous traitons deux tâches de prédiction au moyen de scikit-learn [?] :

- **Classification binaire** — prédire si une panne surviendra dans les prochaines 24 heures. Cette prédiction déclenche une intervention préventive ciblée avant la défaillance.
- **Classification multiclasse (5 classes)** — identifier le type de panne attendu. Cette prédiction oriente le technicien vers la pièce ou le sous-système en cause, réduisant le temps de diagnostic.

L’enjeu industriel est direct et chiffrable. Une fausse alerte mobilise une équipe de maintenance pour rien ; un faux négatif laisse passer une panne réelle qui peut coûter plusieurs heures d’arrêt de production. Cet arbitrage asymétrique justifie le choix de la métrique primaire : le **F1-score** (moyenne harmonique de la précision et du rappel), qui pénalise simultanément les fausses alertes et les pannes manquées.

7.3 Architecture et cycle de vie du modèle

L’architecture du projet suit le cycle de vie standard d’un projet ML, de la donnée brute jusqu’à la mise en service. Chaque étape est isolée dans un module dédié, ce qui garantit la testabilité et la reproductibilité de l’ensemble.



Ce découpage modulaire est un choix d'ingénierie : la préparation, l'entraînement, l'évaluation et le serving sont indépendants. On peut réentraîner les modèles sans toucher à l'API, ou faire évoluer le tableau de bord sans réentraîner. Il est impératif de séparer les responsabilités pour que le projet ML puisse être maintenu.

7.4 Analyse exploratoire et compréhension métier

La maintenance industrielle suit historiquement deux modèles. La maintenance *corrective* répare après la panne : elle maximise l'usage utile des pièces mais expose à des arrêts non planifiés et coûteux. La maintenance *préventive planifiée* révisé à intervalle fixe : elle est sûre mais remplace des pièces encore fonctionnelles, gaspillant de la valeur.

La maintenance *prédictive* vise un troisième modèle, supérieur aux deux précédents : intervenir au bon moment, ni trop tôt ni trop tard, en se fondant sur l'état réel de la machine mesuré par les capteurs. C'est précisément l'objet des deux tâches de prédiction du projet.

Stratégie	Pré-requis	Coût d'une panne	Coût de maintenance
Corrective	Aucun	Élevé (arrêt non prévu)	Faible (minimal)
Préventive planifiée	Calendrier	Faible	Élevé (pièces saines remplacées)
Prédictive	Capteurs + ML	Quasi-nul	Optimisé (juste à temps)

L'indicateur `rul_hours` (*Remaining Useful Life*, durée de vie résiduelle) est au cœur du problème : il estime le temps restant avant la prochaine défaillance. Comme le confirmera l'analyse d'importance des variables, cet indicateur domine la prédiction — un résultat cohérent avec l'intuition métier.

7.4.1 Exploration des données

Le jeu de données regroupe 24 042 mesures de capteurs. L'analyse exploratoire (EDA) précède la modélisation : elle révèle le fort déséquilibre des cibles et la séparation des distributions decapteurs selon le statut de panne, deux observations qui orientent les choix méthodologiques.

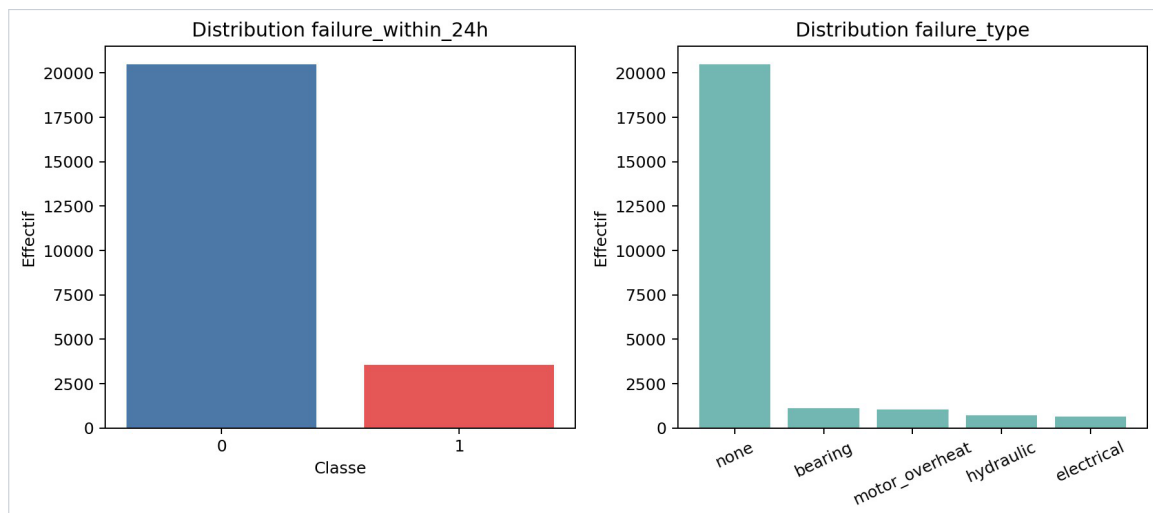


FIGURE 3 – Distribution des deux cibles. La classe positive `failure_within_24h=1` ne représente que 14,8 % des observations, et la cible `failure_type` est dominée par la classe `none` (85 %). Ce déséquilibre impose le *F1-score* et la *PR-AUC* comme métriques, ainsi qu'un *split stratifié*.

Les boîtes à moustaches confirment que les variables dominantes séparent nettement les machines saines des machines à risque : sur les machines en panne, la température moyenne passe de 49 à 63 °C et la durée de vie résiduelle chute de 30,6 à 11,8 h.

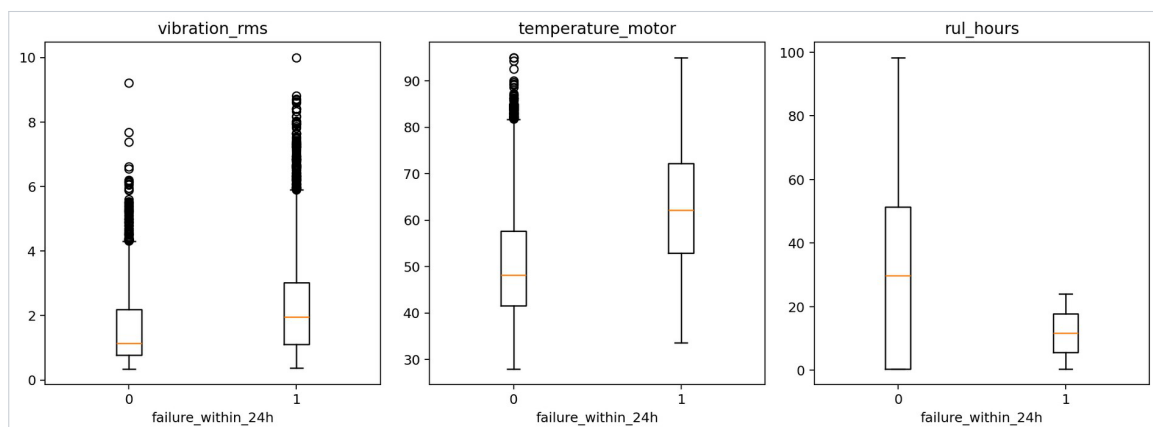


FIGURE 4 – Distribution des variables clés selon le statut de panne. La séparation nette des médianes (température, vibration, RUL) justifie leur pouvoir prédictif observé plus loin.

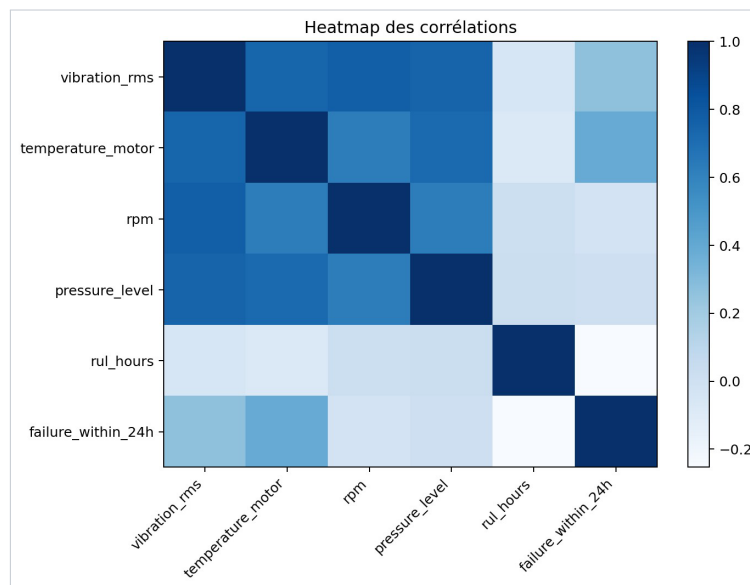


FIGURE 5 – Carte de chaleur des corrélations. La température moteur (+0,39), la vibration (+0,26) et la durée de vie résiduelle (−0,25) sont les plus corrélées à la cible binaire.

7.5 Préparation des données et pipeline de transformation

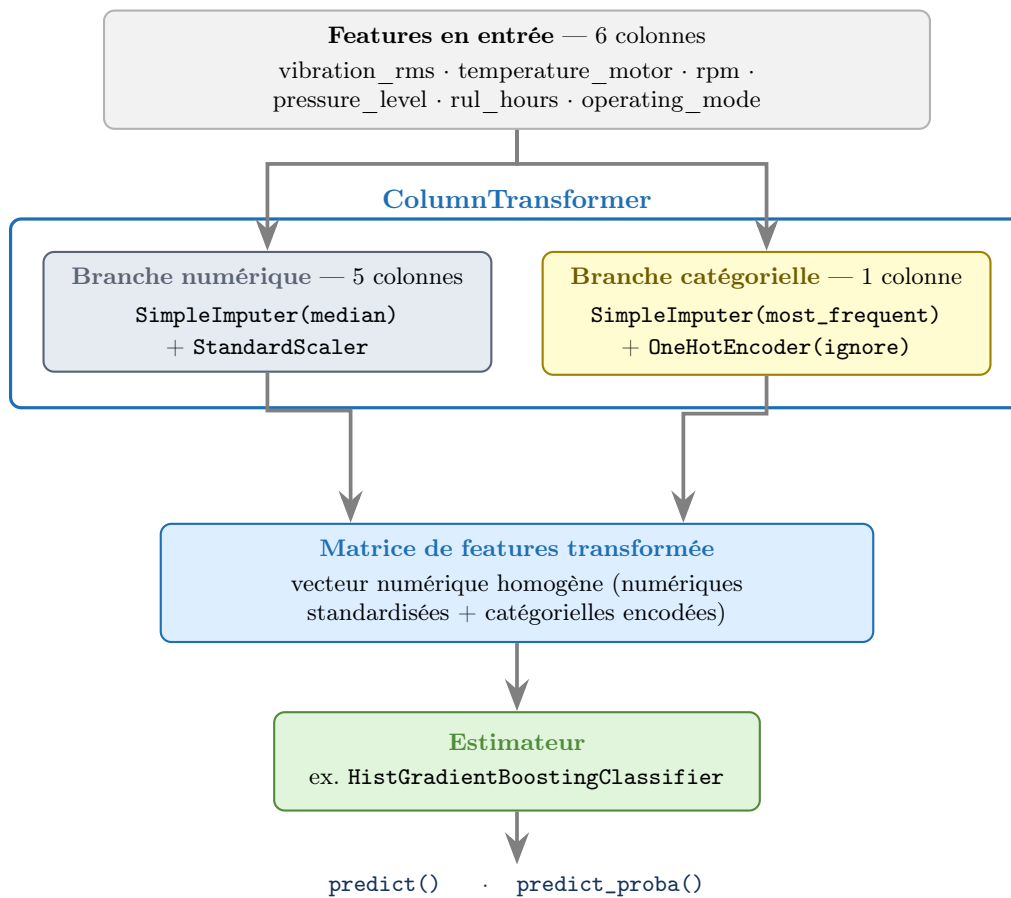
7.5.1 Variables explicatives et cibles

Le jeu de données contient six variables explicatives (cinq numériques, une catégorielle) et deux cibles alternatives, une par tâche de prédiction.

Variable	Type	Description
rul_hours	Numérique	Heures de vie résiduelle estimée
temperature_motor	Numérique	Température moteur (°C)
rpm	Numérique	Vitesse de rotation (tours/min)
vibration_rms	Numérique	Amplitude vibratoire RMS
pressure_level	Numérique	Niveau de pression
operating_mode	Catégoriel	Mode de fonctionnement
failure_within_24h	Cible binaire	Une panne surviendra-t-elle sous 24 h ?
failure_type	Cible multiclasse	Type de panne (5 classes)

7.5.2 Le pipeline de transformation : ColumnTransformer

Les variables numériques et la variable catégorielle exigent des traitements distincts. Le `ColumnTransformer` de scikit-learn applique deux sous-pipelines en parallèle, puis concatène leurs sorties en une matrice de features homogène prête pour l'estimateur. Le schéma ci-dessous illustre ce flux.



```

1 def build_preprocessor() -> ColumnTransformer:
2     numeric_pipeline = Pipeline([
3         ("imputer", SimpleImputer(strategy="median")),
4         ("scaler", StandardScaler()),
5     ])
6     categorical_pipeline = Pipeline([
7         ("imputer", SimpleImputer(strategy="most_frequent")),
8         ("encoder", OneHotEncoder(handle_unknown="ignore")),
9     ])
10    return ColumnTransformer(transformers=[
11        ("numeric", numeric_pipeline, NUMERIC_COLUMNS),
12        ("categorical", categorical_pipeline, CATEGORICAL_COLUMNS),
13    ])
14
15 def make_train_test_split(features, target, test_size=0.2):
16     validate_stratified_split_capacity(target, test_size)
17     return train_test_split(
18         features, target,
19         test_size=test_size, random_state=RANDOM_STATE,
20         stratify=target, # garantit la représentation de toutes les classes
21     )

```

Listing 13 – Pipeline de préparation – ColumnTransformer et split stratifié (preprocessing.py)

Chaque choix de transformation répond à une contrainte du domaine. L'imputation par la **médiane** pour les variables numériques est robuste aux valeurs aberrantes fréquentes sur des capteurs. L'option `handle_unknown="ignore"` de l'encodeur prévient toute erreur à l'inférence si un mode opératoire inconnu apparaît en production : la ligne est encodée par un vecteur nul plutôt que de provoquer une exception. Enfin, la **stratification** du split garantit que chaque classe — y compris les pannes rares — est représentée dans le jeu de test avec la même proportion qu'à l'entraînement, condition d'une évaluation statistiquement valide.

La taille du jeu de test est par ailleurs calculée dynamiquement par `test_size = max(0.2, n_classes / n_rows)`. Ce garde-fou empêche qu'une division 80/20 sur un jeu restreint produise un jeu de test trop petit pour contenir au moins un exemple de chaque classe.

7.6 Entraînement comparatif et sélection du modèle

7.6.1 Architecture du benchmark

Le principe directeur est l'**étanchéité** entre l'entraînement et l'évaluation. Le préprocesseur et le modèle sont encapsulés dans un unique Pipeline scikit-learn : le `StandardScaler` et les imputeurs sont ajustés uniquement sur les données d'entraînement, jamais sur le jeu de test. Cette discipline élimine toute fuite de données (*data leakage*) qui gonflerait artificiellement les scores.

```

1 for model_name, estimator in candidate_models.items():
2     pipeline = Pipeline(steps=[
3         ("preprocessor", build_preprocessor()),
4         ("model", estimator),
5     ])
6     pipeline.fit(x_train, y_train)
7     predicted_labels = pipeline.predict(x_test)
8     predicted_probabilities = pipeline.predict_proba(x_test)
9
10    evaluation_rows.append(evaluate_classifier(
11        task_name=task_name, model_name=model_name,
12        true_values=y_test,
13        predicted_labels=predicted_labels,
14        predicted_probabilities=predicted_probabilities,
15    ))
16
17 comparison = compare_results(task_name=task_name, results=evaluation_rows)
18 best_row = select_best_model(comparison) # tri sur la metrique primaire
19 best_pipeline = trained_pipelines[best_row["model_name"]]
20 save_model(best_pipeline, model_path) # serialisation joblib

```

Listing 14 – Boucle de benchmark et sélection automatique (train.py)

La sélection du meilleur modèle est entièrement automatisée : les résultats sont triés sur la métrique primaire (F1 en binaire, `weighted_F1` en multiclasse) et le vainqueur est sérialisé en `joblib` pour la mise en service. Aucune intervention manuelle n'intervient dans le choix final, ce qui rend la démarche reproductible.

7.6.2 Modèles candidats

Quatre familles de modèles sont mises en concurrence pour chaque tâche, couvrant un spectre volontairement large : un modèle linéaire (Logistic Regression), deux modèles d'ensemble (Random Forest, gradient boosting) et un réseau de neurones (Multilayer Perceptron).

```

1 # Tache binaire (failure_within_24h)
2 {
3     "logistic_regression": LogisticRegression(max_iter=1000, random_state=42),
4     "random_forest": RandomForestClassifier(
5         n_estimators=300, random_state=42,
6         class_weight="balanced"), # compense le desequilibre
7     "gradient_boosting": HistGradientBoostingClassifier(random_state=42),
8     "mlp_classifier": MLPClassifier(
9         hidden_layer_sizes=(64, 32),
10        max_iter=300, random_state=42),
11 }

```

Listing 15 – Modèles candidats par tâche (models.py)

L'option `class_weight="balanced"` de la Random Forest compense le déséquilibre de classes inhérent aux données de maintenance — les pannes sont rares par définition. Le `HistGradientBoostingClassifier` gère nativement les valeurs manquantes, sans imputation préalable, ce qui en fait un candidat robuste même lorsqu'un capteur perd des mesures.

7.6.3 Le modèle vainqueur : `HistGradientBoostingClassifier`

Le `HistGradientBoostingClassifier` est l'implémentation scikit-learn du gradient boosting (approche popularisée par LightGBM). Il se distingue par deux optimisations déterminantes :

1. **Discrétisation par histogrammes.** Plutôt que de tester tous les seuils de division possibles, l'algorithme regroupe chaque variable en 256 intervalles (bins) et ne teste que ces 256 valeurs. La complexité d'entraînement chute de $O(n \times d)$ à $O(256 \times d)$, où n est le nombre d'exemples et d le nombre de variables.

2. **Gestion native des valeurs manquantes.** L'algorithme apprend, pour chaque division, si une valeur manquante doit suivre la branche gauche ou droite. Cette propriété est directement utile face à des capteurs pouvant perdre des mesures (panne capteur, coupure réseau).

Caractéristique	HistGradientBoosting	RandomForest
Valeurs manquantes	Nativement gérées	Imputation requise
Complexité d'entraînement	$O(256 \times d \times \text{arbres})$	$O(n \log n \times \text{arbres})$
Risque de surapprentissage	Modéré (régularisation ℓ_2)	Faible (bagging)
Interprétabilité	Faible (boîte noire)	Moyenne (importance native)
Gestion du déséquilibre	Bonne (<code>class_weight</code>)	Très bonne (<code>balanced</code>)

Comme le montreront les résultats, aucun algorithme n'est universellement supérieur : le Gradient Boosting l'emporte sur la tâche binaire, tandis que la Random Forest domine la tâche multiclasse déséquilibrée. Le benchmark systématique est donc indispensable pour choisir le bon modèle par cas d'usage.

7.7 Résultats et évaluation

7.7.1 Tâche 1 — Classification binaire `failure_within_24h`

Modèle	F1	Précision	Rappel	ROC-AUC	PR-AUC
gradient boosting	0,9775	0,9789	0,9761	0,9991	0,9956
random forest	0,9752	0,9857	0,9649	0,9995	0,9976
mlp classifieur	0,9337	0,9697	0,9003	0,9974	0,9877
logistic regression	0,7224	0,8456	0,6306	0,9485	0,8273

Le **Gradient Boosting** remporte le benchmark sur la métrique primaire F1 (0,9775), avec le meilleur équilibre précision/rappel. La Random Forest obtient une précision et une ROC-AUC légèrement supérieures, mais son rappel plus faible (0,9649) — donc davantage de pannes manquées — pénalise son F1. La Logistic Regression confirme la nature non linéaire du problème : son rappel de 0,6306 signifie qu'elle manque près de 37 % des pannes réelles, ce qui la rend inutilisable pour ce cas d'usage critique.

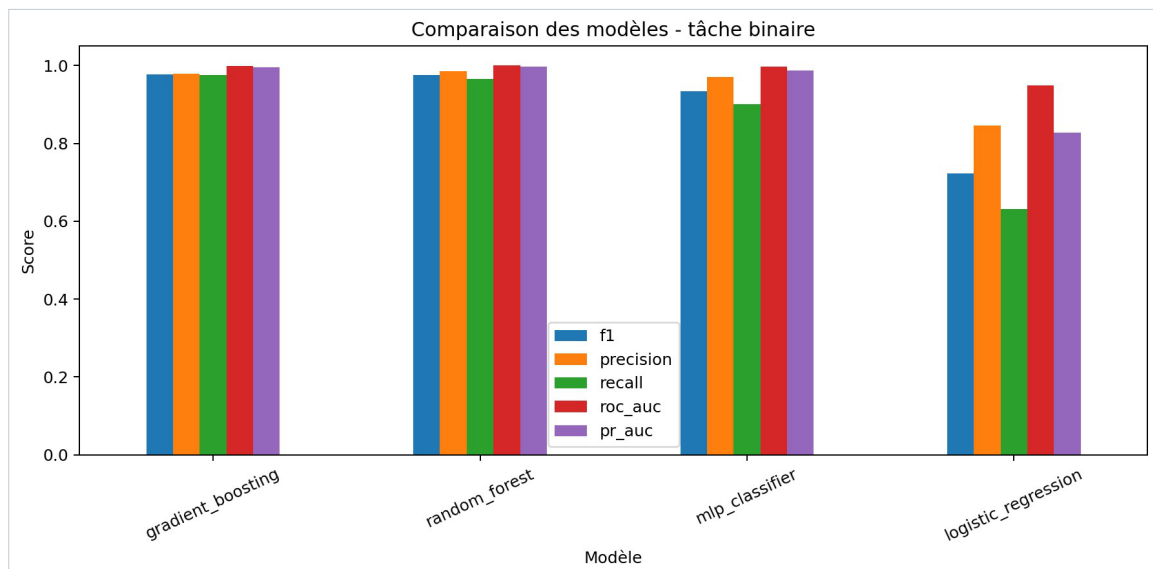


FIGURE 6 – Comparaison visuelle des quatre modèles sur la tâche binaire (F1, précision, rappel, ROC-AUC, PR-AUC). Les deux modèles d'ensemble dominent ; la régression logistique décroche nettement sur le rappel et la PR-AUC.

7.7.2 Tâche 2 — Classification multiclasse `failure_type` (5 classes)

Modèle	weighted_F1	macro_F1	Classes
random_forest	0,9782	0,8955	5
hist_gradient_boosting	0,9753	0,8829	5

Sur la tâche multiclasse, la Random Forest prend la tête (weighted_F1 = 0,9782). Le macro_F1 de 0,8955 — moyenne non pondérée par la taille des classes — indique que les classes minoritaires restent globalement bien gérées. L'écart entre weighted_F1 (0,9782) et macro_F1 (0,8955) révèle néanmoins que certaines classes rares sont prédites avec moins de fiabilité : c'est l'axe d'amélioration principal, détaillé en section [7.9](#).

7.7.3 Interprétabilité — importance des variables

L'interprétabilité est essentielle pour qu'un modèle soit accepté en production par les équipes métier. Nous calculons l'**importance par permutation** sur le jeu de test : pour chaque variable, on mesure la chute du F1 lorsque ses valeurs sont mélangées aléatoirement. Plus la chute est forte, plus la variable est prédictive.

Variable	Importance (permutation)
rul_hours	0,4954
temperature_motor	0,2899
rpm	0,2772
vibration_rms	0,1165
pressure_level	0,0471
operating_mode	0,0174

La variable `rul_hours` domine très nettement (0,495) : elle est le signal le plus prédictif d'une panne imminente, ce qui valide l'intuition métier. La température moteur (0,290) et la vitesse de rotation (0,277) sont les deux facteurs physiques complémentaires. À l'inverse, le mode opératoire est quasi nul (0,017) : il n'apporte pas d'information prédictive significative sur ce jeu de données. Ce classement est directement exploitable : il indique aux équipes quelles grandeurs instrumenter en priorité.

```

1 importance_result = permutation_importance(
2     estimator=best_pipeline,
3     X=x_test, y=y_test,
4     n_repeats=10,          # robustesse statistique de l'estimation
5     random_state=RANDOM_STATE,
6     scoring="f1_weighted",
7 )
8 importance_frame = pd.DataFrame({
9     "feature":      x_test.columns,
10    "importance_mean": importance_result.importances_mean,
11    "importance_std":  importance_result.importances_std,
12 }).sort_values(by="importance_mean", ascending=False)

```

Listing 16 – Calcul de L'importance par permutation (train.py)

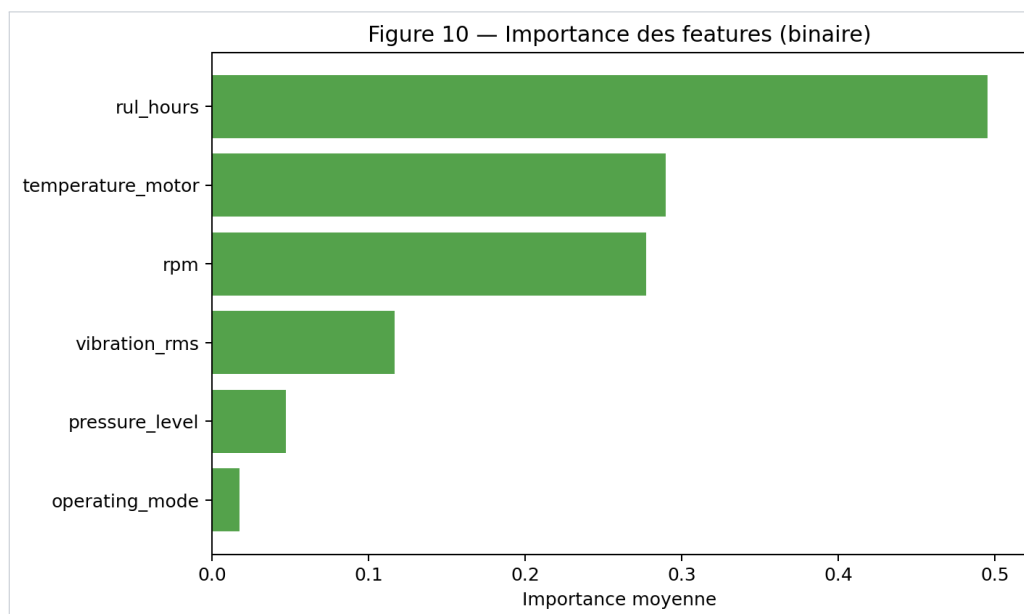


FIGURE 7 – Importance des variables par permutation (tâche binaire). La durée de vie résiduelle `rul_hours` explique à elle seule près de la moitié de la performance du modèle.

7.7.4 Analyse des erreurs — matrice de confusion

Au-delà des scores agrégés, l'analyse de la matrice de confusion éclaire le comportement réel du modèle face aux deux types d'erreurs.

	Prédit : panne	Prédit : pas de panne
Réel : panne	VP (vrais positifs)	FN (faux négatifs — risque métier)
Réel : pas de panne	FP (faux positifs — coût mobilisation)	VN (vrais négatifs)

En maintenance industrielle, les deux erreurs n'ont pas le même coût : un faux négatif (panne manquée) peut entraîner un arrêt de production, tandis qu'un faux positif (fausse alerte) ne mobilise qu'une équipe pour une vérification inutile. Le modèle retenu affiche un rappel de 0,9761 (97,6% des pannes réelles détectées) pour une précision de 0,9789 : l'équilibre penche légèrement en faveur du rappel, ce qui est exactement le comportement souhaité pour ce cas d'usage.

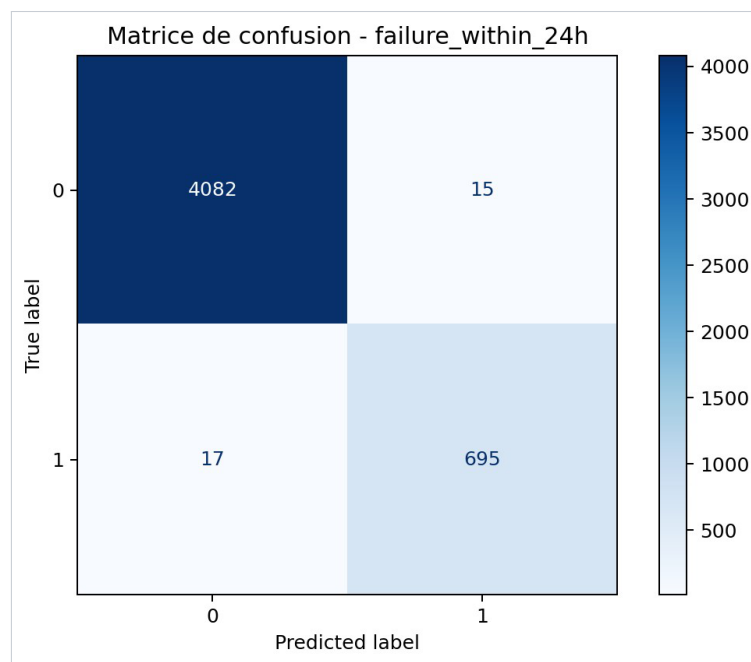


FIGURE 8 – Matrice de confusion de la tâche binaire (modèle *gradient_boosting*). Sur le jeu de test, seuls 17 faux négatifs et 15 faux positifs subsistent face à 4 082 vrais négatifs et 695 vrais positifs.

Sur la tâche multiclasse, la matrice de confusion révèle que la classe majoritaire **none** est presque parfaitement identifiée, tandis que les confusions résiduelles se concentrent entre types de pannes rares (**electrical**, **hydraulic**) — ce qui explique l'écart entre **weighted_F1** et **macro_F1**.

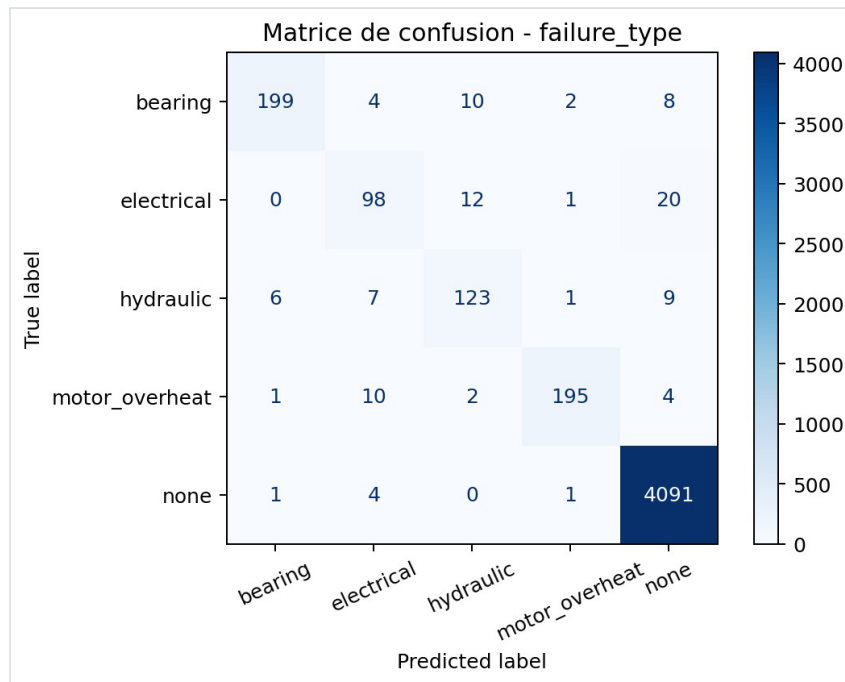


FIGURE 9 – Matrice de confusion de la tâche multiclasse (modèle *random_forest*, 5 classes). La diagonale dominante confirme la qualité globale ; les erreurs se concentrent sur les classes minoritaires.

7.8 Mise en service et communication des résultats

7.8.1 API de prédiction — FastAPI

Les modèles sérialisés sont exposés via une API REST construite avec FastAPI [?]. Chaque tâche dispose de son endpoint de prédiction, et un endpoint de supervision expose l'état de chargement des artefacts.

```

1 @router.post("/predict/failure-within-24h",
2               response_model=FailureWithin24hPredictionResponse)
3 def predict_binary(payload: FailureWithin24hPredictionRequest):
4     return predict_failure_within_24h(payload.model_dump())
5
6 @router.post("/predict/failure-type",
7               response_model=FailureTypePredictionResponse)
8 def predict_multiclass(payload: FailureTypePredictionRequest):
9     return predict_failure_type(payload.model_dump())
10
11 @router.get("/health", response_model=HealthResponse)
12 def health() -> HealthResponse:
13     available_tasks = []
14     for task_name in config.TASK_CONFIGS:
15         try:
16             load_task_bundle(task_name)
17         except TaskArtifactError:
18             continue
19     available_tasks.append(task_name)
20     status = "ok" if len(available_tasks) == len(config.TASK_CONFIGS) else "degraded"
21     return HealthResponse(status=status, available_tasks=available_tasks)

```

Listing 17 – Endpoints de prédiction et supervision (*api/routes.py*)

L'endpoint `/health` renvoie "ok" uniquement si les deux artefacts (binaire et multiclasse) sont chargés ; sinon "degraded". Ce mécanisme permet un monitoring en production. La validation des entrées est assurée par Pydantic v2 via les schémas `FailureWithin24hPredictionRequest` et `FailureTypePredictionRequest`, cohérents avec les six variables déclarées dans `config.py`.

7.8.2 Tableau de bord interactif — Streamlit

Le tableau de bord Streamlit [?] (`dashboard/app.py`) répond directement à la sous-compétence de « communication infographique visuelle ». Il s'organise en trois pages complémentaires :

- **Page exploration (EDA)** : distributions des variables capteurs (histogrammes, boîtes à moustaches), corrélations entre variables (carte de chaleur), répartition des classes cibles. Elle permet à un expert métier de comprendre les données sans écrire de code.
- **Page benchmark** : tableau comparatif des modèles (F1, précision, rappel, ROC-AUC), graphique des importances de variables, sélection interactive du modèle visualisé.
- **Page prédiction** : formulaire de saisie des six variables capteurs, appel en temps réel à l'API FastAPI, affichage de la prédiction et de la probabilité associée.

La page de prédiction ne ré-embarque pas le modèle : elle consomme l'API. Ainsi, tout déploiement d'un nouveau modèle est immédiatement reflété dans le tableau de bord sans le redémarrer — une conséquence directe de l'architecture découplée.

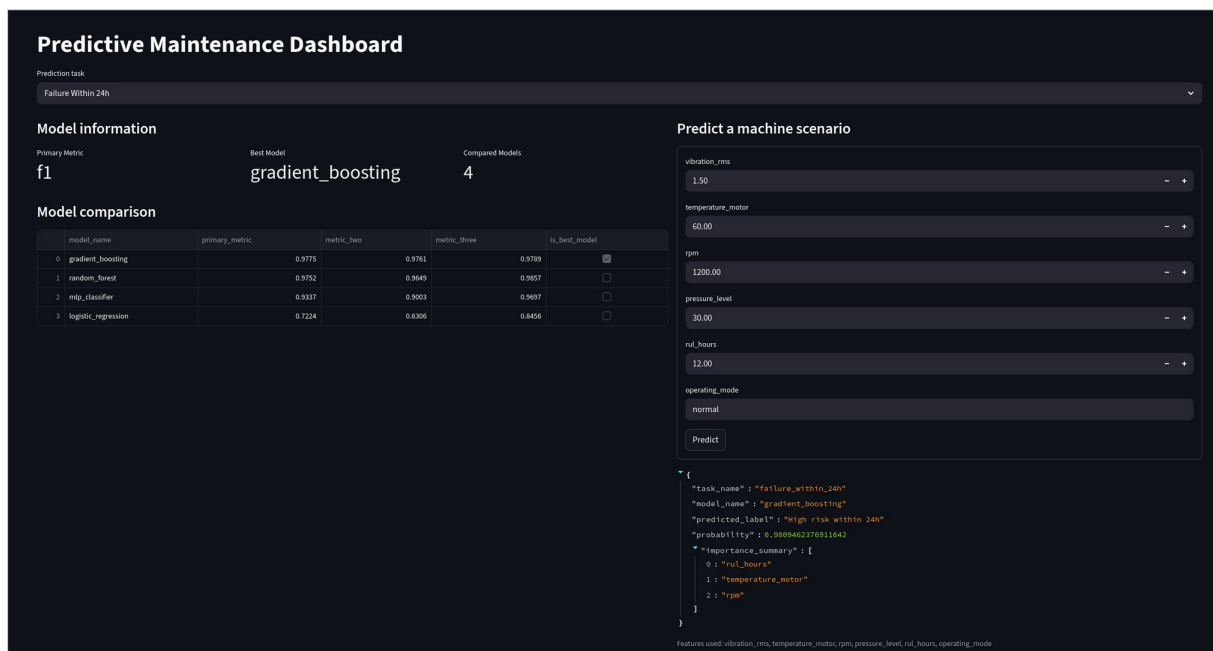


FIGURE 10 – Tableau de bord Streamlit. À gauche : informations du modèle servi et comparaison des modèles. À droite : formulaire de saisie des six variables capteurs et réponse JSON de l'API (probabilité, label, top-3 des variables influentes).

7.8.3 Qualité logicielle et reproductibilité

L'ensemble du pipeline est couvert par des tests automatisés `pytest`, module par module, garantissant que chaque étape est validée indépendamment. Cette couverture est la condition de la reproductibilité des résultats annoncés.

Module testé	Périmètre de validation
<code>test_data.py</code>	Chargement du dataset, séparation features/cible
<code>test_preprocessing.py</code>	Pipeline ColumnTransformer, split stratifié
<code>test_prediction_pipeline.py</code>	Pipeline complet entraînement → prédiction
<code>test_evaluate.py</code>	Métriques binaires et multiclasses
<code>test_explainability.py</code>	Permutation importance
<code>test_api.py</code>	Endpoints FastAPI (health, predict)
<code>test_dashboard_helpers.py</code>	Fonctions de visualisation Streamlit

La configuration des tâches est par ailleurs centralisée dans un unique dictionnaire `TASK_CONFIGS` (`config.py`), source de vérité partagée par les modules d'entraînement, d'évaluation et de serving. Ajouter une nouvelle tâche se résume à y ajouter une entrée : aucun chemin n'est codé en dur dans les modules métier.

7.9 Robustesse, limites et axes d'amélioration

7.9.1 Validation : split stratifié et perspective k-fold

La version actuelle repose sur un split stratifié 80/20, choisi pour son rapport coût/robustesse : huit entraînements complets (quatre modèles $\times$ deux tâches) restent rapides. Une validation croisée *k-fold* ($k=5$) fournirait une estimation plus robuste de la variance, au prix d'un temps de calcul multiplié par cinq.

Méthode	Avantage	Inconvénient
Split 80/20 stratifié	Rapide, simple à implémenter	Variance selon la graine aléatoire
k-fold ($k=5$)	Estimation robuste (k résultats)	$5\times$ le temps de calcul
Stratified k-fold	Robustesse + représentation des classes	Même coût que k-fold

7.9.2 Limites de l'interprétabilité par permutation

L'importance par permutation reste une méthode globale qui présente deux limites. D'une part, en présence de variables corrélées (par exemple `rul_hours` et `temperature_motor`), permuer l'une laisse l'information accessible via l'autre, sous-estimant l'importance des deux. D'autre part, son coût croît avec le produit du nombre de variables par `n_repeats`. Une approche complémentaire serait SHAP (*SHapley Additive exPlanations*), qui fournit une interprétabilité **locale**, instance par instance, en plus de la vue globale.

7.9.3 Axes d'amélioration

L'écart entre `macro_F1` (0,8955) et `weighted_F1` (0,9782) sur la tâche multiclasse indique que les classes de pannes rares sont sous-représentées. Deux leviers sont identifiés : un suréchantillonnage **SMOTE** (génération d'exemples synthétiques pour les classes minoritaires par interpolation) et un ajustement des poids de classe à l'entraînement multiclasse. Enfin, l'ajout d'un réentraînement périodique piloté par une détection de dérive (*drift detection*) permettrait au modèle de s'adapter à l'évolution du comportement des machines en production.

7.10 Synthèse — compétences démontrées

Le projet **data science** couvre l'intégralité du périmètre du Bloc 4, de la donnée brute à la mise en service, en démontrant chacune des cinq sous-compétences du référentiel.

7.10.1 Preuves d'acquisition des compétences

Le tableau suivant récapitule, pour chacune des cinq sous-compétences du Bloc 4, la réalisation concrète qui la démontre dans le projet et le résultat mesurable obtenu.

Compétence visée	Réalisation et preuve dans le projet
Extraire des données de systèmes variés	Chargement et exploration d'un jeu de données de capteurs industriels (<code>data.py</code> , 6 variables, 2 cibles) avec séparation features/cible contrôlée.
Préparer et nettoyer les données	Pipeline <code>ColumnTransformer</code> : imputation médiane + standardisation (numériques), imputation modale + encodage one-hot (catégoriel), split stratifié 80/20.
Communication infographique (tableaux de bord interactifs)	Tableau de bord Streamlit à trois pages : exploration (EDA), comparaison des modèles, prédiction en temps réel via l'API.
Développer un modèle prédictif supervisé	Deux tâches (classification binaire et multiclasse à 5 classes), quatre modèles candidats par tâche, sélection automatisée du meilleur.
Évaluer et comparer les performances	Benchmark sur F1, précision, rappel, ROC-AUC, PR-AUC ; meilleur F1 = 0,9775 (binaire) et <code>weighted_F1</code> = 0,9782 (multiclasse).

8 Bloc 5 – Concevoir une strategie de management et de gouvernance de données pour transformer les données en informations creatrices de valeur

Cette partie doit montrer votre capacite a relier les enjeux techniques aux enjeux organisationnels, reglementaires, qualitatifs et strategiques de la data.

8.1 Competences visees

- Definir une gouvernance de données en mettant en place les politiques et les standards afin d’établir les roles, les responsabilites et la propriete des données.
- Mettre en place les bonnes pratiques de gestion de données en appliquant les regulations en vigueur pour proteger les données et respecter la vie privee dans une logique de transparence.
- S’assurer que la qualite de la donnee permet d’atteindre les objectifs business en faisant un audit de qualite et en proposant des plans de prevention et de remediation en cas de non-qualite.
- Identifier l’approche strategique pour faire de la donnee un actif central ancre dans la culture de l’entreprise et integrer cette approche dans un plan de communication diffuse aux personnes concernees.
- Elaborer une strategie de securite afin de proteger les données en utilisant les technologies adequates et en mettant en place une politique d’accès aux données.

8.2 Projets / TP associes

- Data Management & gouvernance

8.3 Analyse attendue

Mettez en avant les enjeux de gouvernance, de qualite, de securite, de conformite et de communication autour de la donnee, en montrant comment vos travaux transforment la data en valeur pour l’organisation.

9 Conclusion

Concluez en rappelant le fil conducteur du rapport, la progression observée sur les deux années de formation, les compétences acquises et votre capacité à argumenter, justifier vos choix et prendre du recul sur vos réalisations.

Vous pouvez terminer cette conclusion par une ouverture vers la soutenance, qui devra s'appuyer sur un support synthétique et pertinent et mettre en valeur votre e-portfolio de compétences.

Références

- [1] France Compétences. *Fiche RNCP36739 — Expert en ingénierie de données*. Disponible sur : <https://www.francecompetences.fr/recherche/rncp/36739/> [consulté le 15 juin 2026].